

DATABASES · STORAGE ARCHITECTURE · WRITE PATH

One field, five operations

Part I — what a mutation costs: decomposition, the read-modify-write cycle, and the indexed content you pay for on every write

David Leconte — Customer Success Engineer, IBM France | Software, Data & AI | Horizon
Part I of a diptych. Roughly twenty minutes to read; the three exercises at the end take an afternoon between them.

ABSTRACT — PART I OF TWO

Two document databases expose near-identical interfaces — JSON in, JSON out — and diverge sharply in what a single-field update costs. This part argues that the divergence is derivable rather than incidental: it follows from one decision about how a document is laid out on disk, and that decision fixes the price of every mutation thereafter.

A document handed to the decomposing engine is shredded into one row of generic derived columns, flattened to dotted paths at any depth of nesting. Changing one field is therefore not a targeted write but a sequence of five operations, of which a query trace shows the first three directly. Index maintenance on that path covers every indexed field the document carries, not the fields that changed — which an earlier revision of this work stated the other way round, and measurement corrected.

Holding the mutation constant while the document's untouched indexed content grew raised median latency 7.50×, against a read control of 1.56× and a constant network payload. Later campaigns resolved how that price decomposes: the driver is indexed *bytes* rather than field count, and a per-field rate reported in an earlier revision has been withdrawn.

A boundary condition arrived with the answer. Every rate here was measured with the cycle's internal read served from memory; forcing it onto the SSTable path collapses the growth with indexed volume behind a large fixed read cost. The quantities in this part are memtable-regime quantities and are labelled as such wherever they appear. The comparison earlier revisions could not make has also been made: on one host, in one session, under a declared durability mapping, the decomposing engine's cost per mutation runs from roughly three to roughly thirteen times a capability-matched MongoDB's across this series, the gap widening with indexed volume. A ratio of slopes puts it at forty, on a denominator too near zero to bear that weight; the range is what the measurement supports. That is one axis, and it is the axis where decomposition is expected to lose. Two more went the same way for a different reason: through the interface it exposes, the decomposing engine neither aggregates server-side nor counts past a documented threshold, so on the analytical axis the gap is one of capability rather than speed.

Part II takes up the other half of the same decision: when what you wrote becomes visible to a search, why that is a correctness property rather than a performance one, and what a change of consensus protocol does and does not touch.

A note on epistemic status. Comparative writing about competing databases fails most often not through malice but through the silent promotion of plausible mechanism to documented fact. Every substantive claim below therefore carries

one of four markers, and the appendix records the exact passage on which each rests.

D Documented. Established by the vendor's own version-pinned documentation, by the project's source of record, or by peer-reviewed publication. Quoted verbatim in the appendix.

R Reported. Described publicly by the engineers who built the system, or by the upstream project, but not restated in the versioned documentation of the shipped product. Directionally reliable; not a contractual guarantee.

U Unverified. Asserted here on structural grounds, or widely repeated in the practitioner literature, but not established by any source of acceptable rank that I was able to consult. Treat as a hypothesis to be tested against your own build.

M Measured. Observed on a named build under stated conditions, by a harness reproduced in the appendix. Stronger than **U**, weaker than **D**: it establishes what one build did on one day, not what the vendor commits to. Every **M** claim names its measurement (M1–M4) and every measurement names its conditions.

Nota bene on the figures. A diagram carries more apparent authority than a paragraph: it resembles an architecture specification even when it is an argument. Three of the four figures below rest on measurement — two report it directly, one derives intervals from it; all three are tinted green and labelled as such. The fourth is a schematic — they organise an argument and measure nothing, whatever their rectangles may suggest. Every caption additionally states which of its elements are documented, reported or measured. Where a figure depicts something marked **U**, the caption says so, and the box should be read as a hypothesis drawn in rectangles rather than as a contract.

1. The phenomenon

A pattern recurs in design reviews with sufficient regularity to warrant formal treatment.

A team ships a service backed by a document store, and for a while it behaves. Over some months the schema accretes: a status enumeration, two or three timestamps, a nested object carrying audit provenance. Read latency holds. Write latency does not, and it degrades disproportionately to the additional bytes committed. Because no one has altered the write pattern, no one examines the write path. The investigation goes to the application tier, to the network, to garbage collection, and returns empty-handed.

The explanation is neither a defect, nor a tuning deficiency, nor a regression introduced by a release. What it is, is the deterministic consequence of a decision taken long before the team arrived: **how the engine physically represents a document on disk**. Recover that decision, and much of the rest of the system turns predictable, including behaviours with no apparent connection to writes at all, such as the latency with which a newly written document becomes retrievable.

The argument is developed through two systems that resolve the same design question in opposite directions: the JSON document interface layered over a Cassandra-derived engine (the IBM DataStax HCD 2.0 Data API⁵) and MongoDB 8.x.

"Show me your flowcharts and conceal your tables, and I shall continue to be mystified. Show me your tables, and I won't usually need your flowcharts; they'll be obvious."

Frederick P. Brooks, Jr., *The Mythical Man-Month*, chapter 9, "Ten Pounds in a Five-Pound Sack", 1975 (anniversary edition 1995, p. 102).

Written about program data structures half a century ago. This article is an attempt to take it literally, one layer down, where the tables are a vendor's and nobody will show them to you.

2. Two resolutions of the decomposition problem

2.1 Decomposition: one document, one row, many derived columns

The intuitive model holds that a document database stores documents. Some do. This one does something considerably more interesting, and the difference is the hinge of everything that follows.

R A JSON document submitted to the Data API is *shredded*: decomposed into a single database row bearing a generic column layout that captures, separately, the original document text, the set of paths present within it, typed value maps for each JSON scalar type, array membership and array cardinality.¹¹ Indexes are then constructed over the filterable columns.

M The specific column names commonly cited in the practitioner literature (`doc_json`, `exist_keys`, `array_contains`, `array_size`, and the several `query_*_values` maps) are widely repeated but do not appear, so far as I could establish, in any version-pinned DataStax documentation or in a readable source file of the upstream project. I reproduce them below because they are useful as a mental model and because naming them makes the claim falsifiable. They should not be treated as an interface. On HCD 2.0.6 with Data API v1.0.33 (appendix, measurement M1) all nine appear under exactly these names, alongside three that the practitioner literature omits: `key`, the typed partition key; `tx_id`, a timeuuid that guards every conditional write; and `query_lexical_value`, the lexical column. Nine storage-attached indexes were created on that collection, which declared no vector option. That is one build's answer, not an interface; verify against the build you deploy.

M Nesting is where this decomposition earns its keep and shows its edges, and the flat documents of the probes above never touched it. A nested document is flattened to *dotted paths*: every scalar, at any depth, lands in the typed value map for its type, keyed by its full path: `payer.account.id`, `meta.flags.risk`, `lines.0.qty`. A nested field therefore becomes filterable with no index declaration, exactly as a top-level one does.⁴¹ `exist_keys` enumerates every path including array indices; scalar arrays carry membership tokens, which is what makes `{tags: "instant"}` match; `array_size` holds per-path lengths; and `doc_json` keeps the original verbatim.

Two edges are worth knowing before modelling against them. First, inside an *array of objects* only positional paths exist: `{"lines.0.sku": "A"}` filters; `{"lines.sku": "A"}`, meaning "any line with that sku", does not, no path aggregating the field across elements. Second, the shredder imposes hard shape limits: nesting depth at most sixteen, at most one thousand properties per indexable object, and no indexed string beyond eight thousand bytes. All three surface as `SHRED_DOC_LIMIT_VIOLATION` at write time, not at query time, which is the merciful failure mode but still a design constraint rather than a tuning one.

The consequence for what follows is direct. Because a mutation rewrites `doc_json` in full together with every derived map, the write cost of section 4 grows with the *breadth and depth of the nesting*, not only with the count of top-level fields. A deeply nested document is a wide document by another name.

MEASURED ON ONE BUILD

FIGURE 1 — Decomposition and its consequences

One layout decision, four observable properties

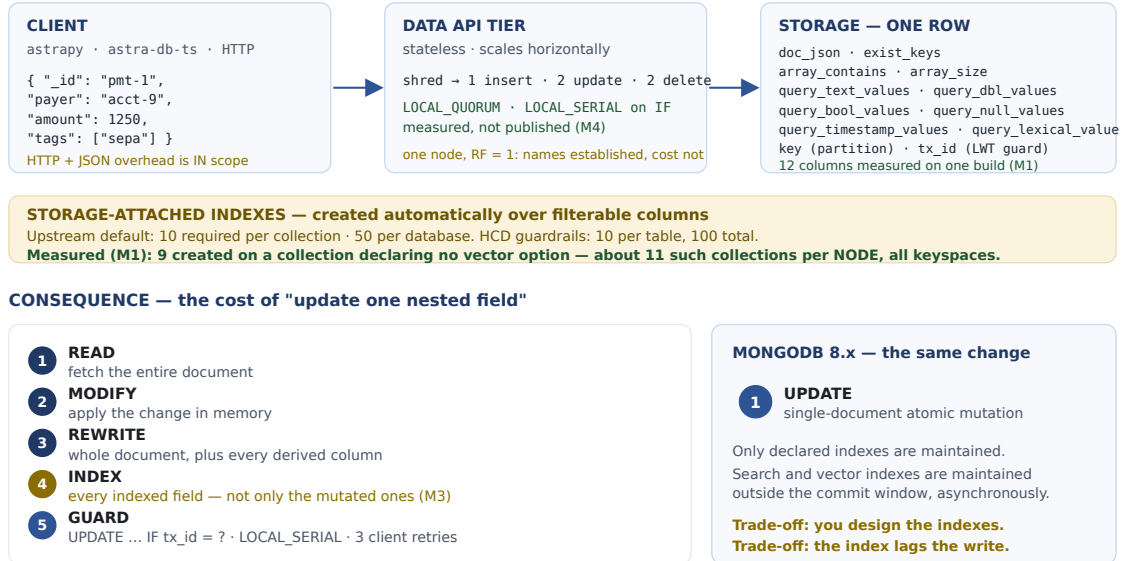


Figure 1. The decomposition decision and the four properties it determines. **Measured, not documented:** the twelve columns and the nine automatic indexes (M1), the statement counts and the read-then-rewrite pair (M2), the scope of index maintenance at step 4 (M3), and both consistency levels (M4). All on one build — HCD 2.0.6 with Data API v1.0.33, one node at RF = 1. The vendor publishes none of it; read the boxes as observations of that build, not as a contract.

D Easily underestimated, and real: any field becomes filterable without anyone declaring an index for it. There is no migration to add a predicate, no ticket to a database administrator to render a new attribute searchable. The upstream project's configuration record fixes the arithmetic: ten indexes are required per collection, against fifty available per database by default¹ — five collections before exhaustion, on the upstream defaults. (That figure is the budget the API checks at creation; the collection measured in the appendix, created without a vector option, received nine — measurement M1.) The shipped HCD product raises the ceiling through a guardrail set to one hundred total storage-attached indexes.³ For a team iterating rapidly on an evolving schema, this is a genuine accelerant.

M It is also the one claim in this diptych that a later comparison confirmed in the platform's favour, so it is worth stating with its number. Against a million documents, filtering on a field nobody had declared an index for: MongoDB as ordinarily deployed scans the collection and answers in 377 ms; the decomposing engine, having indexed that field without being asked, answers in 17.4 ms.⁴⁸ Twenty-two times faster, and the gap is structural rather than tuned: one engine is reading an index that exists, the other is reading everything.

U That median deserves a word, because a scan is not a stable latency. Its cost depends on where in the collection the matching document happens to sit, so the distribution is wide by construction — a median of 377 ms here, against a minimum of 13 ms and a ninety-fifth percentile of 804, with a standard deviation of 258. The figure is an expected value over document placement rather than a measurement of speed, and the honest statement is that scanning a million documents costs somewhere between a few milliseconds and most of a second depending on luck, while the index answers in seventeen regardless. That is the difference worth designing around; twenty-two is a convenient summary of it and not much more.

U The repository's re-analysis puts it more sharply, and against the platform. Because the scan's fastest observation of fifty, 13.3 ms, beats the decomposing engine's median of 17.4 and its slowest of 28.7, the two supports overlap and the direction of this comparison is **not established**: the interval of ratios runs from 0.46 to 65.8 and contains one.⁵² What the data show is not that one is faster than the other but that one is *predictable* — about seventeen milliseconds whatever the document — against a comparator whose answer takes anything from thirteen milliseconds to most of a second depending on where the document sits. Predictability is a real property and a worse headline, and it is the one this row supports.

U The same measurement bounds the claim in the same breath. Declare a wildcard index on the MongoDB side and it answers the same query in 0.73 ms, which is twenty-four times faster than the decomposing engine. The advantage is therefore real against the default and absent against the capability-matched configuration: what is being compared is a default against a decision, not a ceiling against a ceiling. A team that knows this field will be queried can have both the index and the speed on either platform. A team that does not know yet gets the index for free on one of them, and that is the whole of the accelerant.

Everything that follows is the invoice for it.

2.2 In-place mutation

D MongoDB resolves the question in the opposite direction, and its documentation is explicit on all three components of that resolution.

Atomicity. "In MongoDB, write operations are atomic on the single-document level, even if modifying multiple values."³⁴ The guarantee is bounded at the document, and the documentation is equally explicit that it stops there: when a single operation modifies several documents, each modification is atomic but the operation as a whole is not.³⁴ This is a narrower guarantee than the phrase "atomic update" suggests to a reader arriving from a relational background, and the narrowness matters.

Absence of full rewrite. The `$set` reference states the advantage in comparative terms against full document replacement: "Targeted Updates: \$set modifies only the specified fields, ensuring efficient updates by avoiding unnecessary writes and overhead when you work with large documents."³⁵ The same page notes the replication corollary, that `$set` writes only the updated fields to the oplog. The contrast with decomposition is therefore not an inference drawn by this article; it is the distinction the vendor itself draws, between its operator and its own replacement path.

Declared indexes only. MongoDB creates exactly one index without being asked — a unique index on `_id`, at collection creation, which cannot be dropped.³⁶ Every other index is created explicitly. The write therefore maintains what the schema designer decided it should maintain, and nothing else.

D One qualification is owed here, and omitting it would make the comparison less honest than it should be. MongoDB does provide an opt-in analogue of automatic indexing: the *wildcard index*, declared over the pattern `***`, which "can support queries on any arbitrary field within documents in the collection."³⁷ What separates the two platforms on this axis is consequently not *capability* but *default*. One indexes broadly unless instructed otherwise; the other indexes nothing unless instructed. Since defaults govern the systems nobody revisited, the difference is a meaningful one, and it is also smaller and more interesting than "automatic versus manual", which is how the comparison is usually and lazily framed.

"The first principle is that you must not fool yourself — and you are the easiest person to fool."

Richard P. Feynman, "Cargo Cult Science", Caltech commencement address, 1974; published in *Engineering and Science* and hosted by the Caltech library at calteches.library.caltech.edu/51/2/CargoCult.htm.

The section that follows exists because I failed this test, in print, and found out afterwards.

3. On reading published limits — starting with this article's

Before going further into what decomposition costs, a detour that belongs here rather than in an appendix — because the error it describes is one I committed myself, in this article, and did not catch until audit. A reader is entitled to know, early, what this text's claims are worth when they go unchecked.

The upstream open-source project behind the Data API and the shipped commercial product do not carry identical limits, and reading the wrong one silently corrupts a design. **D** The upstream configuration record sets a default read page size of fifty and a maximum insert count of twenty per operation, alongside ten indexes required per collection and fifty available per database.¹ The commercial limits documentation for the same API surface publishes a page size of twenty and a maximum of one hundred insertions per transaction.² The product guardrail for total storage-attached indexes is one hundred.³

Parameter	Upstream project default	Commercial product limit
Read page size	50	20
Documents per insert operation	20	100 insertions per transaction
Indexes required per collection	10	—
Indexes available per database	50	100 total SAI (guardrail)
LWT client retries	3	—

Observe that the divergence does not run in a single direction. Page size is *lower* in the product; insert volume is *higher*; the index ceiling is *higher*. An earlier draft of this article reported the insert batch as 100 upstream against 20 in the product, reversing the direction of the second row and misattributing the third. Three wrong numbers, then: produced by reading the upstream repository where the product documentation was called for, and reproduced with complete confidence until they met the sources now listed in the appendix.

U I should add that I was unable to render the HCD 2.0 limits page itself during verification; the product figures above come from the equivalent commercial documentation for the same Data API surface, which is a source of adjacent scope, not identical scope. The rule this yields is therefore one I am in a poor position to preach and a good position to demonstrate:

Verify the limits that constrain your design against **the build you will actually deploy**: not against a vendor's documentation for an adjacent edition, not against an upstream repository, not against a comparison article, and not against this one. It takes minutes, and it is the difference between a constraint known *ex ante* and one that finds you in production.

4. The cost function of a single-field mutation

Because the document has been taken apart, changing one field cannot be a targeted write. It becomes a sequence:

1. **Read** the entire document into memory.
2. **Modify** it there.
3. **Rewrite** the whole document, together with every derived column.
4. **Maintain** the indexes — on the build measured here, over every indexed field of the document, not only the fields that changed (appendix, measurement M3).
5. **Guard** the write-back with a consensus-based conditional operation, so that a concurrent writer cannot silently overwrite it.

M Steps one through three describe a read-modify-write cycle. This is the structural inference on which the article's title rests, and I must be candid about its evidentiary basis: it follows *prima facie* from decomposition — a derived column cannot be recomputed without the document that derives it — but I could not find it asserted in any DataStax documentation for the current Data API. The one official statement on the subject that I could locate concerns the earlier, architecturally distinct Documents API, and it says the opposite, that the engineers resolved to avoid read-before-write at all costs.¹² Pace that 2020 statement, which concerns a different API generation and cannot settle the question either way. What settles it, for a build you control, is a query trace rather than a latency curve. On HCD 2.0.6 with Data API v1.0.33, with tracing enabled on the node, every `updateOne` carrying a single `$set` produced two CQL statements: a `SELECT` of `key`, `tx_id` and `doc_json` by key, then one `UPDATE` carrying eleven assignments — the nine derived columns, a fresh `tx_id` and the whole `doc_json` — guarded by `IF tx_id = ?` (appendix, measurement M2). The cycle is present on that build. A cost-versus-size probe on the same build grew in step with a same-size read and could not, on its own, separate a read from a read-plus-rewrite; it is the trace, not the curve, that carries the claim.

D Step five is on firmer ground. The upstream configuration record exposes a client-side retry count for lightweight-transaction failure, defaulting to three.¹ The Paxos machinery underlying conditional writes in this engine family is documented explicitly: a prepare phase in which a proposer circulates a proposal number to the replicas of a key, promises returned by a quorum of acceptors, then a proposal of the value, and so forth.¹³ The consistency levels at which this executes in the Data API are **M** not published by any source I consulted; measured on HCD 2.0.6 with Data API v1.0.33, every statement on the collection ran at `LOCAL_QUORUM` and every conditional statement carried `LOCAL_SERIAL` as its serial level (appendix, measurement M4). Earlier drafts asserted `LOCAL_SERIAL` from memory; it is restored here on the strength of a trace.

M A single node at RF = 1 satisfies both levels with one replica, so that first measurement established the names and hid the price entirely. A later run on a six-node ring with three replicas recovered it. Every conditional write executes a full three-replica Paxos round: the coordinator reads the precondition, sends a prepare to the other two, both promise the ballot and return the read row, the

coordinator proposes, both accept and append to commitlog and Paxos memtable, then commit. Coordinator-side median: **13.7 ms for the conditional `UPDATE` against 6.6 ms for a non-conditional `SELECT`** (appendix, measurement M5).³⁹ Roughly double, and that factor is the fifth operation's true cost — invisible on one node, unavoidable on a real deployment, and untouched by any decision about indexing.

The net effect — two CQL statements for an update and for a delete, one for an insert, on the build measured (M2) — is a write materially heavier than a blind CQL insertion, and — importantly — one that is **not idempotent** in the way a blind insertion is. Neither this nor MongoDB's single-step alternative is a defect. Both are the determinate consequence of a storage decision.

4.1 The coefficient the vendor does not publish

M Of the five steps, the fourth is the one this article stated wrongly in every prior draft, and the correction is the most consequential finding in it. The claim was that a mutation maintains "the indexes covering the fields that changed". On the build measured, it does not. It maintains the indexes covering *every indexed field of the document*, whether or not the mutation touched them — which follows inevitably from the trace in the preceding paragraph, since an `UPDATE` that rewrites all nine derived columns necessarily invalidates and rebuilds every index attached to them.

The difference between those two statements is not rhetorical, and it was measured. Holding the mutation constant at one scalar `$set`, and growing only the *indexed content* the document carries — from one such field to seventeen, each of them an eight-thousand-byte string the mutation never touches — median latency rose from 12.1 ms to 90.8 ms, a factor of **7.50**. Over the same range a same-size read by key rose 1.56×, and the HTTP request payload was constant at 122–124 bytes throughout. Neither the network nor the read path accounts for the gap.³⁸

M A multiplier is a poor form for this result, and an earlier revision of this article compounded the problem by converting it into the wrong rate. Running the same document sizes twice — once with the added content left unindexed, once with the same bytes carried as indexed fields — and subtracting at equal byte counts isolates the indexing term from the storage term:

Document size	Update, content unindexed	Update, content indexed	Difference attributable to indexing
8 KB	13.4 ms	15.7 ms	2.3 ms
32 KB	14.9 ms	32.1 ms	17.3 ms
128 KB	22.1 ms	90.8 ms	68.7 ms

That run could not say what *shape* the term has, because it grew indexed field count and indexed bytes together — each ballast field being an eight-thousand-byte string. A revision of this article nevertheless divided the difference by the number of added fields and reported roughly four milliseconds per indexed field. The protocol did not license that division, and the rate was withdrawn. A later campaign was designed to license it.

4.2 Which term carries the cost

M The separation cannot be done with three arms at one document size, which is worth stating because it is the design most people reach for first. The engine refuses any indexed string above eight thousand bytes, so at 128 KB of indexed content the *minimum* field count is seventeen — the reference arm itself. "Few long fields" does not exist at that size. Two series are needed instead, each holding one term fixed.⁴³

Series	Held constant	Varied	Median update	Growth	r ²
Bytes vary	16 indexed fields	512 → 8000 B per field	25 → 115 ms	×4.6	0.9999
Fields vary	64 KiB indexed content	9 → 128 fields	67 → 75–80 ms	×1.13 – ×1.19	0.57 / 0.92

Multiplying the field count by fourteen without changing a single byte moves the median by thirteen to nineteen per cent. Multiplying the bytes by sixteen at a fixed field count moves it by a factor of four and a half, along a line so straight that the least-squares slope and the endpoint ratio agree to three

significant figures in both passes. **The cost is carried by indexed bytes.** Field count is a real but secondary term.

Which restores the rate the earlier revision had no right to, in the unit the experiment actually supports: on this build, a mutation costs about **0.76 to 0.79 milliseconds per kilobyte of indexed content** the document carries and the mutation never touches — plus roughly 0.08 to 0.11 ms for each additional indexed field.

M That rate has a boundary condition, and a later probe found it. Every measurement behind it, including the ones taken in a regime this article previously called disk-bound, had its read served from the memtable: the read-modify-write cycle re-reads the row it has just written, and that row is memtable-resident however large the dataset around it. Forcing the internal `SELECT` onto the SSTable path instead — by flushing between the write and the mutation — changes the result qualitatively.⁴⁶

Document size	Read served from memtable	Read forced onto the SSTable path
1 KB	20.4 ms	107.8 ms
128 KB	121.2 ms	163.4 ms
growth	×5.94	×1.52

A large fixed cost appears, roughly ninety to a hundred milliseconds of read path. That alone would reduce the ratio without touching the underlying rate, and an earlier revision of this section said only that the growth "disappears behind it". The stronger reading is the correct one: the absolute rise across the series falls from 100.8 ms to 55.6 ms, so the per-byte term itself is *halved* on the SSTable path rather than merely dwarfed by a constant. **The per-kilobyte rate is therefore a memtable phenomenon in both senses.** Where the read leaves memory, it is the read path that decides the cost of a mutation, not the quantity of indexed content. The mechanism of section 4.1 is unaffected: the cycle is still a read, a rewrite of everything, and an index maintenance pass. What does not transpose is the number.

U Two reservations bound that in turn, and they run in opposite directions. Flushing before every mutation produces one SSTable per flush, so the internal read merges an unnaturally fragmented set — up to thirty-odd small tables — which a normal compaction schedule would not leave lying about. The fixed cost above is therefore an upper bound, and a realistic disk-bound figure sits somewhere between the memtable runs and this worst case. In the other direction, the operating system page cache could not be dropped for want of root, so "SSTable path" still does not mean "cold platter". A third reservation is one of provenance rather than method: the post-flush record does not state its topology or replication factor, while the memtable figures it is set against come from a three-replica ring. Until that is reconciled, the contrast between $\times 5.94$ and $\times 1.52$ may mix the read path with a change of deployment — the same confound this article objects to elsewhere, and it would be dishonest to object to it there and not here. The honest statement is that the rate is bounded below by a memtable regime and above by a fragmented-read one, and that no measurement in this study occupies the middle.

M Two passes, thirty timed repetitions per configuration after five discarded warm-ups, on a three-replica ring in a proven disk-bound regime.⁴³ The design carried a free internal control: one configuration — sixteen fields of 4 096 bytes — belongs to both series and was therefore measured twice per pass. The two readings agreed to 2.0 % and 0.7 %, which is what licenses the slopes above; had they disagreed by more than ten per cent, nothing in this subsection would be reportable.

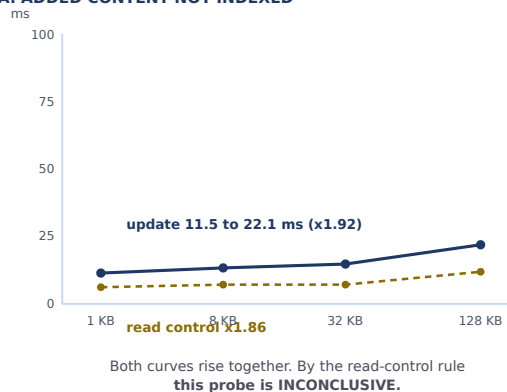
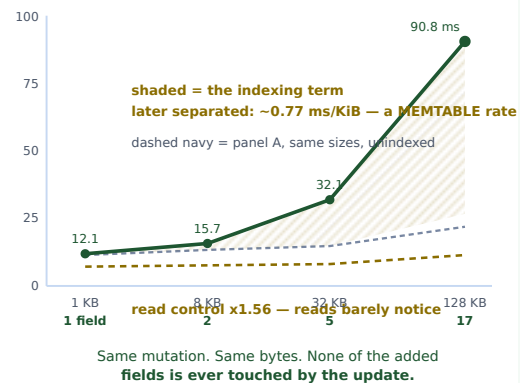
U The strict verdict rule, declared before the run, asked for a fields-series endpoint ratio at or below 1.15 to call bytes dominant. The two passes came in at 1.128 and 1.187 — they straddle the threshold, so the pre-registered verdict is formally *inconclusive*. I report that rather than quietly adopting the favourable pass. Neither passes approaches the 1.5 that would make field count an independent driver, and the direction is not in doubt; the honest statement is that bytes dominate and that the exact size of the residual field term is not pinned down.

The practical consequence is a change of question. A design review should not ask how many indexed fields sit on the mutation path. It should ask **how many kilobytes of indexed content** do — which is a different quantity, arrived at differently, and the one you are actually charged for.

MEASURED ON ONE BUILD

FIGURE 2 — Isolating what indexing costs a mutation

One scalar \$set, held constant. Identical document sizes in both panels. Only whether the added content is indexed changes.

A. ADDED CONTENT NOT INDEXED**B. SAME BYTES, CARRIED AS INDEXED FIELDS**

Each ballast field is an 8 000-byte indexed string, so field count and indexed bytes rise together: the difference is the indexing term, but its per-field and per-byte sh

Figure 2. **Measured, not documented** (appendix, measurement M3). Panel A is why the earlier draft could not settle its own claim: unindexed growth moves update and read together. Panel B holds the byte sizes identical and carries the same content as indexed fields; the shaded area between the two update curves is the indexing term. **Confound not eliminated:** each ballast field is an 8 000-byte indexed string, so field count and indexed byte volume were grown in lockstep; this run isolates the cost of indexing content rather than merely storing it, but separates neither a per-field nor a per-byte rate. A later campaign did separate them — see section 4.2 — and found the byte term dominant at about 0.77 ms per indexed kilobyte. **Regime:** every point here was taken with the cycle's internal read served from memory, a condition section 4.1 shows to be decisive — forcing that read onto the SSTable path halves the per-byte term behind a large fixed cost.

On this build, the price of a mutation is set by **how wide the document is indexed**, not by how much of it you changed. Automatic indexing is charged per write, per indexed field, forever — not once, at insertion.

Three consequences follow, and they are design consequences rather than tuning ones.

Adding a field is a write-latency decision. The opening phenomenon of this article — latency creeping upward while the write pattern never changed — now carries a coefficient. A team that widens its schema is buying write cost on every mutation of every document of that shape, including mutations that never go near the new field.

Collection density is bounded by arithmetic, not by taste. M A collection created with default options and no vector dimension received nine storage-attached indexes,³⁸ against platform guardrails of ten per table and one hundred in total — the latter a ceiling *per node, across every keyspace*, not per database.³ That gives roughly eleven such collections per node before the guardrail refuses — and a collection declaring a vector option consumes a tenth index, arriving exactly at the per-table threshold.

Selective indexing stops being an optimisation and becomes, for some payloads, compulsory.

M The Data API rejects any *indexed* string longer than 8 000 bytes.³⁸ A document carrying a description, a transcript or a chunk of source text cannot be stored at all unless that field is excluded from indexing at collection creation. The mechanism that reduces the coefficient is therefore already in your hands, and for large text you are obliged to reach for it.

Which is the practical upshot, and it is worth stating without hedging: *decide* what is indexed, per collection, at creation. The clause costs one line. The default costs 7.50× on the mutation path of a wide document, and costs it silently.

4.3 Where that cost is actually paid

M A rate tells you what a mutation costs. It does not tell you *which component* spends it, and the two are different design objects. The work described above is done in two places: the Data API tier parses the JSON, shreds it into eleven derived values and builds the statement pair, all of which scales with indexed bytes; the storage engine then executes the pair and maintains the indexes, which also scales with indexed bytes. Nothing in the rate distinguishes them.

Separating them takes a subtraction. Run the same mutation through the Data API, then run the identical CQL statement pair directly against the same row with the shredded values read back rather than recomputed: the storage work is the same, the tier work is absent, and the difference is the tier.⁴⁵

Term	Slope (ms per indexed KiB)	r ²	Share of the rate
Stateful storage engine	0.547	0.997	≈ 71 %
Stateless Data API tier	0.219 (passes: 0.190 and 0.248)	0.98	≈ 29 %
Sum	0.772	—	the total, by construction: the tier term <i>is</i> total minus storage

U That last row proves nothing, and an earlier revision of this section said it did. The tier term is *defined* as the total minus the storage term, so the two sum to the total by construction, and least squares being linear the identity carries to the slopes. I wrote that "nothing forced that agreement". Arithmetic forced it. The claim is withdrawn, and it is worth recording that the error ran in the flattering direction: the section congratulated itself on a verification it had not performed.

M There is a real replication in the same numbers, and it is not that one. The *total* slope was measured at 0.763 and 0.790 ms per indexed kilobyte in one campaign, on one keyspace; it came back at 0.772 in a later campaign, on a different keyspace, on a different day, through a different instrument.⁴⁵ Nothing forced *that* agreement, and it is the closest thing to an independent confirmation the study contains.

U A second, independent estimate of the tier — subtracting coordinator-side durations from client-observed latency, using query traces rather than a bypass — puts it lower, at 0.145 ms per KiB and nineteen per cent. The two methods agree to within a quarter at 8, 16 and 32 KiB and diverge beyond it, by 34 % at 64 KiB and 28 % at 125 KiB. They subtract different things: the bypass removes a complete client-side CQL execution, the trace removes only the coordinator's internal duration. A second dispersion compounds the first, and it sits *inside* the bypass method rather than between methods: its two passes returned tier slopes of 0.190 and 0.248, thirty per cent apart, which is wider than the quarter this study treats as disqualifying between instruments. The coefficients of determination above describe the fit within a pass, not the agreement between them. **No single precise split is therefore published here.** What survives both methods is the direction and the order of magnitude: the tier is a substantial minority, the storage engine the majority.

Which matters for a reason that has nothing to do with milliseconds. **Between a fifth and a third of this cost sits in a component that scales out by adding instances. The rest sits in one that does not.** The criticism this article makes of automatic indexing is therefore softened in part — and only in part, the centre of gravity staying where it was.

U A later measurement destabilised the split further, and in the direction that costs the platform. Adding a third arm to the comparative probe — HCD driven straight through CQL, the tier bypassed entirely, on the same host and the same series — put the slope at 0.704 ms per KiB against 0.777 through the Data API. Removing the tier removed **nine per cent** of the rate, not the twenty-nine per cent the subtraction above reports.⁴⁷ The two estimates differ threefold and were taken under different regimes and interleavings, and neither rests on separated supports: at every size, the Data API and direct-CQL ranges intersect. What the pair establishes is that the tier is a minority; where in the range from a tenth to a third it falls is not settled, and this article should not be read as settling it. The practical consequence points one way: on the later, cleaner measurement, roughly nine tenths of the cost sits in the component that does not scale out.

U A further dispersion sits in the subtrahend itself. The direct-CQL arm is markedly noisier than the arm it is subtracted from: at 8 KiB its median is 17.9 ms against a ninety-fifth percentile of 61.9, and at 64 KiB its minimum is 14.3 against a median of 49.3. A distribution that wide is not plausibly unimodal, and subtracting its median from a cleaner one yields a tier term whose uncertainty this study does not quantify. The direction survives that; a decimal place does not.

U One further caveat governs the absolute numbers and not the conclusion. The tier was measured against a container with two gigabytes of memory and *no CPU limit at all*, on a host with eighty cores. A tier sized for production would behave differently, and the absolute latencies above would not transfer. The question asked here was where the rate lives, and a slope answers that robustly; the same measurement would not support a capacity plan.

4.4 The comparison, measured at last

M Every earlier revision of this work carried the same reproach in its limitations: measurements on one side, documentation on the other. That is no longer true. The identical mutation series was run against both engines on the same host, in the same session, with the same repetition counts, under a declared durability mapping — and the harness refuses to emit a ratio when those conditions are not met.⁴⁷

Arm	Slope (ms per indexed KiB)	r^2	Median at 8 KB → 128 KB
MongoDB, nothing indexed beyond <code>_id</code>	0.0098	0.78	5.7 → 6.5 ms
MongoDB, wildcard index over every field	0.0176	0.88	5.5 → 7.6 ms
HCD, engine only, tier bypassed	0.704	0.996	18.4 → 101.6 ms
HCD, through the Data API	0.778	0.997	40.9 → 130.8 ms

Neither MongoDB arm alone is the fair comparison, and the pair is the finding. The first is MongoDB as a team would deploy it, indexing nothing it was not asked to. The second is MongoDB doing this platform's job, every field queryable without declaration — the closest analogue to automatic indexing that MongoDB offers.

U Two quantities follow from that table and they are not the same size, so both belong here rather than whichever is the more striking. The ratio of *slopes*, **tier removed** against the capability-matched arm, is forty on the pass the harness fitted — and 116 on the pass it did not, which the paragraphs

below take up. **U** An earlier revision called this ratio “engine to engine”. It is not: every HCD update in this dossier, through the interface and through direct CQL alike, is emitted as a conditional statement, so removing the tier removed the tier and left the consensus round standing. The term for what the ratio compares is *tier removed*, and the cost of consensus is identified nowhere in this work. The ratio of *latencies* is smaller and grows with the document: 3.3× at 8 KB, rising to 13.3× at 128 KB. A reader told “forty times” will hear the second quantity and receive the first, which is why this section declines to lead with it.

U Two facts about that table were established after it was drawn, and both belong beside it. The slopes were fitted on the first of the two passes collected; the merge step of the harness, which the author of this article wrote, read pass one only, and nothing in earlier revisions said so. The second pass exists in the repository and was not used. And a re-analysis of the raw summaries by separation of support — asking, for each document size and pass, whether every observation of one arm beat every observation of the other — finds the mutation comparison **disjoint in nineteen of the twenty size-and-pass cells**: at those sizes the direction is certain without any distributional assumption, with an exact one-sided permutation probability of about 10^{-17} at thirty observations a side.⁵² The same re-analysis finds the tier-removal comparison — the Data API against direct CQL — **overlapping at every one of its ten cells**, so the nine per cent of section 4.3 is a difference of medians inside supports that intersect, and is not an established direction.

U The slope ratio has no defensible value at all, and saying why takes a paragraph rather than a caveat. A ratio of two fitted slopes is bounded only when the denominator is distinguishable from zero, and MongoDB’s is barely that on the pass the harness used and not at all on the pass it ignored. Fitting both passes and carrying the standard errors through:

Denominator slope, ms per indexed KiB	Pass one	Pass two
MongoDB, wildcard index	0.0176 ± 0.0038	0.0063 ± 0.0026
MongoDB, nothing declared	0.0098 ± 0.0030	0.0085 ± 0.0058
Ratio to the decomposing engine, wildcard arm	44×	116×
Interval on that ratio, by Fieller’s theorem	[26, 142]	unbounded

On the second pass the denominator’s own interval reaches zero, so the ratio has no finite upper bound — a division by something not distinguishable from nothing. The point estimate moves by a factor of 2.6 between two passes of the same probe on the same machine. **What is defensible is that the latency gap runs from roughly three-fold to roughly thirteen-fold across this series, and that it widens with indexed volume because one engine’s cost scales with that volume and the**

other's barely does. That statement rests on medians and ranges, which the evidence supports; the slope ratio rests on a fit that does not survive its own second pass, and an earlier revision of this section published it as a number.

U One consequence reaches beyond this paragraph. The separation-of-support re-analysis that certifies five of the eight axes operates on pairs of samples and performs no fit; it therefore says nothing about any slope, and the four ratios most likely to be quoted from this work — 44×, 79×, 40×, 72× — fall outside everything that apparatus establishes. They are the least supported figures in the diptych, not the best.

On the mutation path the direction is not close, and no reading of the conditions rescues it. What decomposition costs here is a property of what decomposition is, measured rather than argued. The magnitude is a range, and it depends on how much indexed content the document carries.

D One feature of the mapping deserves naming, because it runs against the conclusion and survives anyway. MongoDB was held to a majority write concern with journalling, which waits for the journal to reach disk before acknowledging; HCD ran with a periodic commit-log sync, which acknowledges first and syncs within ten seconds. The durability terms therefore favoured HCD — it was allowed to answer before the disk had heard about the write, and MongoDB was not. It lost across the whole range regardless. An objection that reverses into a strengthening is worth more than one that merely fails.

M A companion campaign extended the comparison to reads, and the result is not favourable. Fetching one document by identifier from a million: MongoDB answers in 0.53 to 0.62 ms, the decomposing engine in 10.4 ms.⁴⁸ Roughly twenty times, on a path where no shredding, no rewrite and no consensus round occurs.

U The obvious reading is that this ten-millisecond floor is the Data API tier, and an earlier revision of this section asserted it. The measurement does not support the assertion. What was compared is an HTTP interface against a native driver, and no direct-CQL read arm was run — the bypass of section 4.3 covered mutations only, where it attributed nine per cent to the tier, not ninety. The gap is measured; its allocation between the tier, the driver and the engine is not, and one arm would settle it.

M Eight axes have now been measured across the two parts, and the summary belongs here rather than scattered through both.

Axis measured	Which engine	Margin, and its reservation	Direction, by separation of support	Beyond the measured envelope?
Mutating one field	MongoDB	3–13× in latency; a memtable-regime figure. The slope ratio is 44× on the pass fitted and 116× on the pass ignored, unbounded there by Fieller	certain — disjoint in 19 of 20 cells	Holds — the cost follows from decomposition itself; more nodes distribute it, none removes it. <i>Unmeasured:</i> under sustained concurrent writes, on either side
Fetching one document by identifier	MongoDB	≈ 20×; allocation between tier and engine unmeasured	certain — disjoint	Undetermined — much of the gap is the interface's HTTP hop, never measured on the read path; it does not depend on volume
Filtering on a declared or wildcard-indexed field	MongoDB	≈ 24×	certain — disjoint	Undetermined — measured at one million documents, cache-resident on both sides

Axis measured	Which engine	Margin, and its reservation	Direction, by separation of support	Beyond the measured envelope?
Filtering on a field nobody declared	<i>not established</i>	$\approx 22\times$ at the median, and only against the default configuration; what is established is predictability, not speed	not established — supports overlap, ratio interval contains one	Undetermined — a scan degrades with the collection and an index much less, but a direction not established inside the envelope cannot be claimed to establish itself outside it
Freshness of an ordinary secondary index	neither	index current on both; elapsed time is not	tie on attempts; elapsed time disjoint both ways	Undetermined — one datacentre only. Across sites the <code>LOCAL_ONE</code> asymmetry would become observable, <i>against</i> the decomposing engine
Aggregating by group	MongoDB	a capability gap in the <i>interface</i> , not a ratio between engines: no server-side	certain — disjoint, on three observations	Holds, and worsens — an absent verb arrives at the first document, and the client

Axis measured	Which engine	Margin, and its reservation	Direction, by separation of support	Beyond the measured envelope?
		pipeline; 134 s against 0.2 s by the only route left. The engine's own doctrine says to pre-compute the aggregate at write time, which is neither route		scan grows linearly: 10 000 round trips at 200 000 documents, 500 000 at ten million
Counting exactly	MongoDB	by capability: the interface caps exact counts at a documented threshold, and its estimator answered zero before the first flush	structural — no observations, the operation is refused	Undetermined — the refusal is constant, but the comparator's cost depends on the route: its unindexed scan grows with volume while its indexed count, measured at 12.7 ms, does not

Axis measured	Which engine	Margin, and its reservation	Direction, by separation of support	Beyond the measured envelope?
Freshness of the search index (Part II)	Decomposing engine	total inside the comparator's refresh window, void outside; that window measured $\approx$ 1 s here	certain by the miss-rate test; not established as a latency comparison	Undetermined — the boundary is the refresh interval of a single-node development edition; dedicated search infrastructure may refresh faster or less often

U One caveat governs the table as a whole and is easy to miss because the rows read as commensurable. They are not. The mutation row comes from a three-replica ring in a cache-resident regime; the read and filter rows from a million-document collection against MongoDB 8.0 in a three-member replica set; the aggregation and counting rows from a two-hundred-thousand-document corpus on the same pairing; the search-freshness row from MongoDB 8.3.11 in a single-member deployment, through a different image. Four configurations, one table. The repository's register lists two further measurements with no comparator — where the mutation cost is charged, and vector freshness under a single-replica read — which are properties of one engine rather than axes of comparison and are not counted here. Each row is defensible against the row it replaced in an earlier revision; no row is a controlled comparison against the others, and a reader should not compute across them.

U One column of that table needs its own paragraph, because it decides what a reader may carry away. The measurements were taken on one shared host, at two hundred thousand to one million documents, by a single sequential client, cache-resident but for one campaign, in one datacentre. **That envelope is the one in which the decomposing engine is weakest and the comparator strongest:** nothing that makes the case for a distributed engine — partitioning at scale, sustained concurrent writes, multi-site replication — was exercised once. Three regimes could change the picture and none was measured. When the dataset outgrows one node's memory, where the comparator's unindexed count degrades and its indexed one does not. When write load outgrows what the deployment absorbs — though the comparator here ran as a three-member replica set with a single primary, and a sharded

one would not. And across datacentres, where the consensus round measured here at its minimum would cost tens of milliseconds and the single-replica vector read would become observable, *both against the decomposing engine*.

Five of the eight go to MongoDB. One goes to the decomposing engine, on the axis its write-path indexing exists to serve. One goes to neither. And one — filtering on a field nobody declared — goes to nobody at all: its median favours the decomposing engine by twenty-two times and its supports overlap, so the article names no winner where it cannot establish one. No engine wins them all, and a reader who wanted a verdict will have to settle for a shape.

U What this does not establish is as important as what it does, and it is a narrow set of axes. The series measures one thing: mutating one field of a document carrying unchanged indexed content. It says nothing about filtered reads, vector search, aggregation, or the interval between a write and its visibility to a query — which is the axis where this platform has its structural advantage, and which Part II could only bound below a measurement floor. A study that measured only the unfavourable axis would be as biased as a complacent one. This one has measured the unfavourable axis well and the favourable axis not at all, and the reader should weight it accordingly.

4.5 Two axes the interface does not have

M Everything measured so far has compared two ways of doing the same operation. A seventh campaign found two operations one side cannot do at all through the interface it exposes, and on that axis a comparison is not a ratio but a capability gap.⁵⁰

Aggregation. The Data API's command allow-list on this build holds nineteen commands.

`aggregate`, `$group` and `distinct` are not among them and answer `COMMAND_UNKNOWN`. There is no server-side aggregation pipeline. Summing a field by category over two hundred thousand documents therefore has exactly one route: fetch every document to the client and add them up there. Measured, that route took a median of **134 seconds**; MongoDB's server-side `$group` took 223 ms over the same corpus, both results verified exact against precomputed ground truth.

U That is a factor of six hundred — **M** at *ten groups*, and the qualifier is not decoration: re-measured at three query shapes, the ratio is 584× at ten groups, 151× at a hundred thousand, and 609× filtered, on disjoint bootstrap intervals. The comparator's cost follows group cardinality and the interface's does not, being dominated by transport, so the ratio falls by a factor of 3.9 across that range. The direction holds everywhere; the magnitude is a property of the query chosen, and it must not be read as the storage engine being six hundred times slower, because the same engine on the same host aggregated the same corpus in about two seconds through native CQL. The 134 seconds belong to the interface, and this diptych's own numbers say how: the vendor documents a page size of twenty (section 3), so the scan is ten thousand round trips, and the decomposition needs no constant from elsewhere: 133 984 ms over ten thousand pages is **13.4 ms per page**, with no residue to explain. An earlier revision decomposed it instead with the 10.4 ms point read of section 4.4 — a different operation, on a different collection, from a different campaign — and produced a thirty-second remainder that was an artefact of the imported constant rather than a component of anything. A documented limit multiplied by a per-page cost the measurement itself yields; nothing a reader can tune away. And the CQL route is not an escape for the document model: reaching it means writing CQL against a declared schema whose shape is fixed at creation — abandoning the document model,

which is structural and needs no percentile. Reaching it also means designing a table partitioned by the group key in advance. An earlier revision withdrew that clause, on the ground that the table designed for the query and a cross-partition `GROUP BY` against that same table differ by only $\times 1.28$ at the median, with overlapping supports and a ratio interval containing one.⁵² **U** The withdrawal was an error of object rather than of arithmetic: both figures come from *one* table, and what separates them is the shape of the query, not the design of the schema.

M A probe written to settle it shows why. Against a table whose primary key does not carry the group column, that second query is not slower — the engine refuses it outright, with or without a filtering hint.⁵² The clause stands, and with one limit its own evidence imposes: the *per-partition sweep* that produced the two-second figure does run against an unpartitioned table once `ALLOW FILTERING` is added — 44.0 ms against 14.3 ms aligned, on a two-thousand-row probe table, at a cost never measured at the scale of the headline. What is categorically inexpressible without pre-partitioning is therefore the `GROUP BY` arm alone, not the arm that carries the number.

Counting. **D** The vendor documents that exact counting beyond a server-side threshold is not supported by the Data API and recommends the estimate instead.⁵¹ **M** On this build the threshold is a thousand: both an unfiltered and a filtered count of the same corpus refused with `TooManyDocumentsToCountException`, on a stock container. The recommended estimate answered in 7 ms and returned **zero** for a collection holding two hundred thousand documents. It reads SSTable metadata, and before the first flush there is none. **M** An earlier revision called that a cold-start property and noted the steady state unmeasured. It has since been measured, and the reading was too generous: after a flush the estimate returns 171 267, after a major compaction 172 132, against 200 000 real documents — an error of about fourteen per cent that does not converge, on a corpus where the comparator’s estimator returns 200 000 exactly. The zero is the extreme of a permanent defect, not a transient one. The application testing `if count == 0` on a freshly loaded collection does not know it is in a cold start, and nothing in the response tells it.

D Before that lands as a verdict, a question of fairness has to be settled, and settling it changes what the row means. Is it legitimate to measure an engine on an operation its own doctrine tells you not to perform? The doctrine is explicit and long-standing: this engine supports no complex processing at query time — no filtering, no joins, no search, no aggregation — and the data must be stored partitioned, sorted, denormalised and filtered in advance, so that ideally all that is left is to read the answer from a single location.⁵³ The modelling corollary follows from it: store *pre-computed answers* to the questions the application will ask, and expect roughly a table per query. Reading many partitions to assemble one answer is, in the vendor community’s own words, tolerable for two to five partitions and bad for the cluster beyond that.⁵⁴

So the architecturally correct answer to “sum this field by category” on this engine is **neither** of the two routes measured. It is a table of aggregates maintained at write time, which reads in milliseconds. Measuring 134 seconds against 223 milliseconds compares two answers to a question that one of the two engines asks you not to pose that way — and a reader who takes the 602 as a statement about engines has taken it wrongly — and one who takes it as a statement about aggregation in general has taken it wrongly twice, since it is a statement about aggregation *at ten groups*.

U What keeps the row legitimate is that the subject measured is not the engine but the *interface*, whose entire proposition is document ergonomics in the manner of the comparator: the same verbs, the same shape of query. A team that adopts it because it resembles the comparator will expect `aggregate`, and will discover its absence at a `COMMAND_UNKNOWN` rather than in any document. The comparison is therefore fair at the level of the API surface and unfair at the level of the storage engine, and both halves have to travel together. The honest form of the finding: **a team that needs ad-hoc aggregation is on the wrong engine, and a team that believes it has adopted a comparator-compatible API needs to know that this one is not, on this axis.**

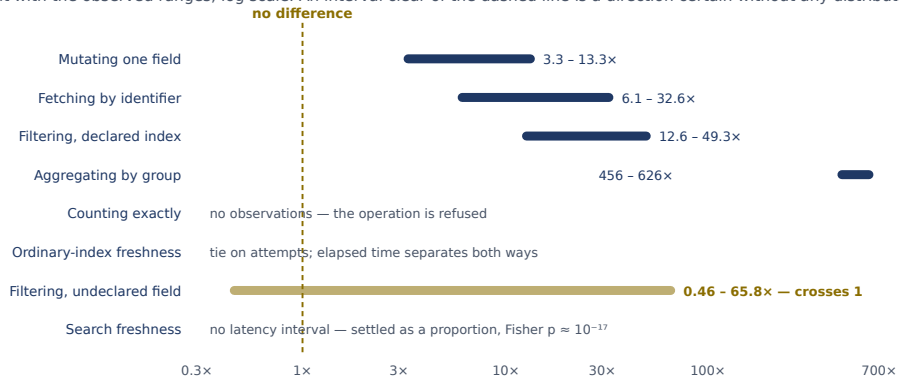
On the analytical axis the finding carries no percentile. The interface does not aggregate, does not count past a threshold it declares, and its estimator answers zero before the first flush. Three sentences, and they survive everything this diptych has learned to break.

U What this does not establish: anything about the Data API's table mode, which was not tried on this axis; anything about an analytical engine attached to the same ring, which many deployments have; and anything beyond one build, since an allow-list is a product decision that can change between versions. The measurement stands in the cache regime — the zero from the estimator proves the memtable had not been flushed — and the client-side scan was timed three times, which is enough for a six-hundred-fold effect and for nothing subtler.

DERIVED FROM THE RAW EVIDENCE

FIGURE 3 — The eight axes, and which of them establish a direction

Interval of ratios consistent with the observed ranges, log scale. An interval clear of the dashed line is a direction certain without any distributional assumption.



Five axes favour the comparator on intervals clear of one. The two that favour the decomposing engine do not appear as established latency intervals: one crosses one, the other is not a latency question at all. That asymmetry is the honest shape of this study's result.

Figure 3. **Derived, not measured** — computed from the summary statistics in the repository by separation of support (appendix, note 52). Each interval is [slower arm's minimum ÷ faster arm's maximum, slower arm's maximum ÷ faster arm's minimum], the widest ratio the observed ranges permit; no distributional assumption is made and none is available, since every probe but one discarded its per-observation series. **Read with three cautions.** The rows come from four deployment configurations and are not commensurable with one another. The aggregation interval rests on three observations against fifteen. And an interval clear of one establishes a direction, never a magnitude a reader can plan with.

4.6 Two habits that follow directly

The difference surfaces in two places under your control: how you write, and what identifier you assign the record.

```
# astrapy — the Python client for the Data API
#
# (1) Server-generated identifier: convenient, and NOT safe to retry.
# If the network fails after the write applied, a retry creates a second
collection.insert_one({"payer": "acct-42", "amount_cents": 125000})

# (2) Identifier derived from the business key: a retry is rejected as
# so the operation becomes naturally idempotent. Same cost, far better
collection.insert_one({
    "_id": "pmt-2026-08-07-000123", # from the payment reference, not
    "payer": "acct-42",
```

```
"amount_cents": 125000,
})
```

That single choice eliminates an entire category of production incident, on either platform, at no runtime cost. It is the cheapest correctness improvement available in this class of system.

The second habit is to let the engine apply the change rather than performing it yourself:

```
# Anti-pattern: read-modify-write in application code.
# Three round trips, and two concurrent workers can silently lose one u
doc = collection.find_one({"_id": "pmt-...000123"})
doc["status"] = "SETTLED"
collection.replace_one({"_id": doc["_id"]}, doc) # rewrites everything

# Better: transmit the intent, not the document.
# One round trip. Whatever the engine does internally, it does it once,
# atomically, and without shipping the document across the wire twice.
collection.update_one(
    {"_id": "pmt-2026-08-07-000123"},
    {"$set": {"status": "SETTLED"}},
)
```

The opening phenomenon now resolves itself. Write latency crept upward because the team *added fields*, and each added field was silently enrolled in the maintained index set. Nothing about their write pattern changed. The cost of their write pattern did.

5. What decomposition costs, and the design rule that follows

The cost localises in one identifiable regime: **workloads that repeatedly mutate the same record**. Counters. Status machines transiting a sequence of states. Hot aggregates updated on every event. There, the in-place update is structurally cheaper, and no quantity of tuning alters the structure.

Model for append and read at scale, not for repeated mutation of the same record.

If your access pattern genuinely concentrates writes on individual records — and some legitimately do — that is valuable information about which engine suits that component. It is far cheaper to learn in week one than in month six, and cheaper still to learn from a storage diagram than from an incident review.

D It bears stating explicitly, since the term "transaction" recurs throughout this literature with three mutually incompatible senses, that the DataStax documentation is unambiguous about the guarantees on offer *sensu stricto*: HCD does not adhere to ACID transactions with rollback or locking mechanisms, and a write operation is atomic at the partition level.³³ Section 9 examines what is changing upstream on exactly this point.

"There are two ways of constructing a software design: One way is to make it so simple that there are obviously no deficiencies, and the other way is to make it so complicated that there are no obvious deficiencies. The first method is far more difficult."

C. A. R. Hoare, "The Emperor's Old Clothes", 1980 ACM Turing Award lecture, *Communications of the ACM*, vol. 24 no. 2, February 1981, p. 77.

Automatic indexing is the second kind of simplicity: nothing to declare, and therefore nothing obviously wrong.

6. Three design rules

6.1 Keep partitions bounded, and know the numbers

D A brief detour, because it matters here. The platform offers two data models side by side: schemaless *collections*, which is what everything above has described, and schema-defined *tables* carrying an explicit primary key.⁶ Collections suit documents whose shape varies; tables suit records of stable shape read by key and range — ledgers, event trails, time series. Both are reached through the same JSON API; neither requires dropping into a query language.⁷

M With one caveat that matters before you design around them: on the build measured, `createTable` answers `TABLE_FEATURE_NOT_ENABLED`.⁴² Tables are behind a flag that is off by default in Data API v1.0.33, in addition to carrying a public-preview designation. Treat them as a deliberate configuration choice to be made and validated, not as a capability already sitting in your deployment.

D Where tables are used, the published sizing guidance is explicit and arithmetically actionable: keep rows per partition under one hundred thousand and partition size under one hundred megabytes.⁸ The corroborating modelling guidance states the second bound independently.⁹ Note that the documentation specifies *rows*; a document decomposed to one row makes the two coincide here, but the distinction survives elsewhere and is worth preserving in your own notes.

The partition key determines how rows are grouped, and a key that accumulates without bound breaches both limits eventually:

```
// A table's primary key is declared in JSON, as part of createTable.
// partitionBy -> how rows are grouped and distributed
// partitionSort -> the ordering within a partition (1 ascending, -1 descending)

// UNBOUNDED: one partition per account, growing for the life of the account
// Flawless in test. Degrades reads, repairs and streaming in year two.
"primaryKey": {
  "partitionBy": ["account_id"],
  "partitionSort": { "event_ts": -1 }
}

// BOUNDED: the month joins the partition key, so each partition covers
// fixed window. A query spanning months reads a handful of partitions
// rather than one enormous one.
"primaryKey": {
```

```
"partitionBy": ["account_id", "yyyyymm"],
"partitionSort": { "event_ts": -1 }
}
```

Choose the window from your own arithmetic: the busiest account's events per month, multiplied by bytes per event, held beneath both bounds above.¹⁰ This single decision forestalls the majority of the difficulties popularly attributed to this class of database.

FIGURE 4 — Bounding the partition key

The same data, two groupings: one degrades monotonically, the other does not

partitionBy: ["account_id"] — one partition per account, growing for the life of the account

account_id = acct-42

Jan · Feb · Mar · Apr · May · Jun · Jul · Aug · Sep · Oct · Nov · Dec · Jan · Feb · Mar ...
unbounded row count · unbounded byte size · breaches both published limits eventually

Flawless in test.

Degrades reads, repairs
and streaming in year two.

partitionBy: ["account_id", "yyyyymm"] — one partition per account per month, of fixed maximum width

202601

bounded

202602

bounded

202603

read

202604

read

202605

read

202606

bounded

202607

bounded

A quarterly query reads
three partitions, not one
unbounded one.

Published bounds, both of which the left-hand design breaches in time: under 100,000 rows per partition, and partition size under 100 MB.

Figure 4. Partition bounding. **Documented:** both numeric bounds (notes 8 and 9) and the JSON key structure (note 10). **Illustrative:** the monthly window, which is one choice among many; the correct granularity follows from your own arithmetic, not from this figure.

6.2 Choose what gets indexed, deliberately

Automatic indexing is a default, not a law. If a field will demonstrably never be filtered upon, excluding it renders every write to that document cheaper. On the other platform the same discipline applies *mutatis mutandis*, inverted in its direction: declare the indexes you need, and no more. In both cases the discipline is the same act — deciding, rather than inheriting.

6.3 Measure write-to-searchable before designing around it

It is twelve lines, and virtually nobody runs it:

```
import time, uuid

marker = f"probe-{uuid.uuid4()}"           # unique, so nothing else can n
t0 = time.perf_counter()
```

```
collection.insert_one({"_id": marker, "body": marker})

while True:                                     # poll until the write is find
    if collection.find_one({"body": marker}):
        print(f"write-to-searchable: {(time.perf_counter() - t0) * 1000}")
        break
    time.sleep(0.01)
```

Run it idle, then run it again under write load. Two things this probe is not. It does not tell you whether an update reads before it writes — for that, enable query tracing on one node for a handful of updates and read `system_traces`, the method behind measurement M2. And, on evidence, it does not cleanly measure index lag either: what it returns is an interval that *contains* any index lag alongside two network round trips, and on the build measured here the lag term was zero and the transport term was all of it. Record the *attempt count*, not just the elapsed time — a document found on the first query has no measurable lag, whatever the stopwatch says, and a rising attempt count is the only unambiguous signal that an index is behind. The second run is the one that matters, because it is under load that the two architectures diverge and under load that your users will encounter whichever behaviour you inherited. A quarter of an hour establishes whether index freshness is a constraint you must design around or one you may safely ignore. If you are building anything with retrieval in it, this is the highest-yield fifteen minutes in this article.

7. What to do this week

Both apply whatever database you run today, and neither takes longer than an afternoon.

1 — Inventory the indexed content on your mutation path. Thirty minutes.

For each entity your service updates frequently, write down how many *kilobytes* of it are indexed — the field count matters, but far less than the volume inside those fields, and nested paths count too. That quantity, not the size of your change, is what you pay on every write. On the build measured, roughly 0.77 ms per indexed kilobyte per mutation; your own number will differ and is worth having. Where it is large and the mutable subset is small, the remedy is one clause at collection creation on the decomposing engine, or one fewer `createIndex` on the other.

2 — Establish who generates your record identifiers. One hour.

Walk the writes that matter — payments, orders, anything on which a duplicate would be expensive — and determine whether the identifier derives from your business data or from the database. Convert the ones that matter. This eliminates a category of production incident, costs nothing at runtime, and behaves identically on every database in this class.

3 — Ask your own database how it stores a record. An afternoon.

Whatever you operate, establish whether it keeps a record whole or takes it apart, and what it indexes automatically. Then re-read your slowest write path in that light. In the majority of cases the explanation is already there, and the investigation has been examining the wrong layer.

8. Conclusion to Part I

What this part establishes. A document handed to the decomposing engine is not stored as a document. It is shredded into one row of twelve columns — nine derived value maps, the document text, a partition key and a transaction identifier — over which nine storage-attached indexes are created without anyone asking. Changing one scalar field is therefore not a targeted write: a query trace shows a `SELECT` of the whole row followed by an `UPDATE` carrying eleven assignments, guarded by a conditional on the transaction identifier. Nesting is flattened to dotted paths, so depth and breadth both enter the cost. These are direct readings of the engine's own schema and trace, reproduced across passes; they are the strongest evidence in this diptych, and the vendor publishes none of them.

What follows, with its coefficient. Index maintenance on a mutation covers every indexed field the document carries, not the fields that changed — an earlier revision of this article stated the reverse, and measurement corrected it. The cost rises with *indexed content the mutation never touches*, and the driver is indexed **bytes** rather than field count: at constant volume, multiplying the field count fourteenfold moved the median by thirteen to nineteen per cent; at constant field count, multiplying the bytes sixteenfold moved it by a factor of four and a half along a line of near-perfect fit. The rate measured about 0.77 ms per indexed kilobyte.

And the boundary of that coefficient, which matters as much as the coefficient. Every rate here was taken with the cycle's internal read served from memory. Forcing that read onto the SSTable path halves the per-byte term and introduces a fixed cost of roughly a hundred milliseconds that dominates it. The rate is a memtable-regime quantity. The *mechanism* — read, rewrite everything, maintain every index — is regime-independent; the *number* is not, and no figure in this part should be carried into a disk-bound design without being re-measured there.

Against the comparator, on this axis, the decomposing engine loses, and the loss is structural. On one host, in one session, under a durability mapping that favoured it, and with the stateless tier subtracted from both sides of the argument, mutating one field costs between three and thirteen times more, the gap widening with indexed volume. Fetching one document by identifier costs about twenty times more. Only about a tenth of the mutation disadvantage lies in the stateless tier; the rest is the engine. This is what the layout decision predicts, and it is the price of what the layout buys.

What it buys, measured. On a million documents, filtering on a field for which nobody declared an index costs 377 ms against a comparator that must scan, and 17 ms here. That is a real operational advantage, and it is an advantage against a *default* rather than against a ceiling: the same comparator, given a wildcard index, answers in 0.73 ms. What the platform offers is not speed but the absence of a design decision — you never pay for the index you forgot to declare.

What this part does not establish. A disk-bound read without the artificial fragmentation the measurement introduced. Concurrency, multiple clients, multiple hosts. Aggregation. The internal decomposition of the tier's own cost. Any confidence interval: thirty repetitions and two passes give shapes, not precisions.

8.1 The same result, arranged by workload

The table in section 4.4 is arranged by what was measured. This one is arranged by what a reader has. Each row names a workload profile, the engine the measurements favour for it, and the condition under which that preference holds — because every one of them has a condition, and a row read without it is a row misread.

U Do not count the rows. They are profiles, not points, and they do not tally with the eight axes above: three of MongoDB's axes are folded into two rows here, one of the decomposing engine's axes is split across two, and one row for the decomposing engine rests on an inference rather than a measurement. Read row by row, each is defensible. Summed, they would suggest a parity the measurements do not show — the score, if a reader wants one, is the table in section 4.4: five axes, one, one to neither, and one the data do not settle. The last column says which measured axis each row rests on, and with what evidentiary status.

If your workload is...	Choose	On what evidence, and under what condition	Rests on
Write then search, composed within the refresh window a step that writes and a step that reads, with no human and no model in between	Decomposing engine	The only unconditional win in the diptych: 100 % of documents missed against 0 % at zero delay, still one in two at half a second. <i>Condition:</i> your application's write-to-search delay must be shorter than your deployment's refresh interval. Both are measurements you must take.	M search freshness, inside the window
Counters, status machines, hot aggregates the same record mutated repeatedly	MongoDB	Three to thirteen times cheaper per mutation, widening with indexed volume, and roughly nine tenths of the disadvantage is the engine rather than the interface. <i>Condition:</i> a memtable-regime figure; the gap changes shape once the read leaves memory.	M mutation
Read-dominated, latency-sensitive fetch by key, filter on known fields	MongoDB	About twenty times cheaper by identifier, about twenty-four on a declared or wildcard index. <i>Condition:</i> the split of that gap between the stateless interface and the engine was not measured on the read path.	M point read <i>and</i> declared-field filter — two axes, one row

If your workload is...	Choose	On what evidence, and under what condition	Rests on
Exploratory, schema still moving fields queried before anyone decided to index them	Decomposing engine	Twenty-two times faster than a comparator that must scan a million documents. <i>Condition:</i> the advantage is against a <i>default</i> . Declare a wildcard index on the other side and it is twenty-four times faster still; what you are buying is the absence of a decision, not speed.	M undeclared-field filter
Append-dominant with retrieval over it written once, read and searched many times	Decomposing engine	Index maintenance is charged once, at insertion, and the search index is current at acknowledgement. <i>Condition:</i> this is an inference from two measured properties, not a separately measured workload, and it carries no number of its own.	U inferred; no axis
Conversational, human-paced or model-paced anything where seconds pass between writing and reading	MongoDB	The freshness advantage vanishes entirely once the read arrives after the refresh window, and the mutation and read advantages remain. <i>Condition:</i> none — this is the case the diptych previously got wrong, and correcting it removed the platform's claim on this profile.	M search freshness, outside the window — the other half of row one

If your workload is...	Choose	On what evidence, and under what condition	Rests on
Analytical — aggregate, group, count anything that needs a sum, a group or an exact cardinality server-side	MongoDB	The interface has no aggregation pipeline and caps exact counts at a documented threshold; the only route pulls the collection to the client, 134 s against 0.2 s. <i>Condition:</i> through the Data API as exposed. An analytical engine attached to the same ring, or the engine's native query language over a table designed for the query, were not the object measured.	M aggregation <i>and</i> counting — two axes, one row
Read your own writes through an ordinary index	neither	Both maintain ordinary secondary indexes synchronously; the index is current on both. Elapsed time differs, but that measures the query rather than the index.	M ordinary-index freshness

U Four caveats govern every row. The rows come from three different deployment configurations and are not commensurable with one another. Everything was measured on one host, one build of each engine, with almost no regime outside the cache. Nothing was measured under concurrency, and production is concurrent. And the profiles named in the first column are descriptions rather than measurements: no real application's write-to-search interval was timed, so a reader recognising their own system in a row is making a judgement the study cannot make for them.

Establish how your database physically stores a record and you can predict most of what it will cost you. That claim survives seven campaigns, including the five axes it predicts unfavourably for the platform this author works on. What it does not do is favour an engine.

Part II takes up the other consequence of the same layout decision — when what you wrote becomes findable, why that is a question of correctness rather than speed, and the narrow condition under which it decides anything at all.

Disclosure. I work at IBM, on the DataStax side of Software, Data & AI. IBM also sells the comparator: MongoDB Enterprise Advanced with IBM is an IBM offering, generally available and supported by IBM,⁴⁹ so my employer has no commercial interest in one of these two engines prevailing over the other. What is not neutral is my knowledge: I know one of these systems considerably better than the other, and that asymmetry — of expertise, not of interest — is the one the reader should weight both parts of this diptych by. Two deliberate constraints follow. **First**, no benchmark figures appear in the body of either part, because a number produced on my hardware with my workload would tell you nothing about yours; the measurement appendix reports one estate's observations, each with its conditions, and none of them licenses a comparison with anything. **Second**, every factual claim is graded for evidentiary status and sourced — including claims favourable to my employer, claims I was unable to verify, and two that measurement showed to be wrong.

Corrections are welcome and will be applied with attribution. The apparatus exists so that an error can be demonstrated in minutes rather than argued about, and it is public: every raw measurement record, every probe, the two unmodified originals against which the patched ones can be diffed, the architecture decision record, and the adversarial audits written against this work are at github.com/davidleconte/shredding-cost-hcd-mongodb.

David Leconte — Customer Success Engineer, IBM France · Software, Data & AI | Horizon

Notes

Note numbers run continuously across both parts of this diptych, so a reference keeps its identity wherever it is cited. This part uses 21 of them, and the gaps below belong to the other part rather than to an omission. The appendix that follows carries, for every note, the full locator, the passage relied upon where one exists, and any caveat of scope or edition.

1. Stargate, `data-api`, `CONFIGURATION.md`, branch `main`. Establishes `default-page-size` = 50, `max-document-insert-count` = 20, `indexes-needed-per-collection` = 10, `indexes-available-per-database` = 50, and `lwt.retries` = 3. Upstream project defaults; not the shipped product's limits.
2. DataStax, Data API limits, Astra DB Serverless. Page size 20; maximum insertions per transaction 100. Commercial documentation for the same API surface; the HCD 2.0 limits page could not be rendered during verification.
3. DataStax Mission Control, HCD guardrail configuration: `sai_indexes_total` set to 100.
4. DataStax, *Cassandra drivers overview*, HCD 2.0: "HCD 2.x is based on Cassandra 5.0."
5. DataStax, tables reference, HCD. Tables via the Data API carry a public-preview designation at the time of writing.
6. DataStax, HCD frequently asked questions: the Data API is described as schema-less and document-based, providing access to both structured and unstructured data.
7. DataStax, capacity planning, HCD: under 100,000 rows per partition and partition size under 100 MB.
8. DataStax, data-modelling best practices, CQL for HCD: partitions ideally under 100 MB.
9. DataStax, list table metadata, Data API. Source of the `primaryKey` / `partitionBy` / `partitionSort` structure quoted in section 6.1.
10. Stargate project, introduction of the JSON API and Mongoose integration, 13 March 2023. Source of the "super shredding" characterisation; does not enumerate column names.

11. Stargate project, the Cassandra Documents API, 19 October 2020: the engineers state that they resolved to avoid read-before-write operations at all costs. Concerns the earlier Documents API, architecturally distinct from the current Data API.
12. DataStax, lightweight transactions, HCD. Describes the Paxos phases underlying conditional writes.
13. DataStax, how transactions differ, HCD: HCD does not adhere to ACID transactions with rollback or locking mechanisms; a write operation is atomic at the partition level.
14. MongoDB, *Atomicity and Transactions*, Database Manual. Single-document atomicity, and its explicit boundary for multi-document operations.
15. MongoDB, `$set` update operator reference, Database Manual. Targeted updates against full document replacement; oplog efficiency.
16. MongoDB, *Indexes*, Database Manual. The unique `_id` index created at collection creation, which cannot be dropped; all other indexes created explicitly.
17. MongoDB, `createIndexes` reference, Database Manual v8.0. Wildcard indexes over the `$**` pattern, supporting queries on arbitrary fields.
18. Measurement record, single-node campaign: `findings.json`, run 2026-09-17T14:59Z, one node at RF = 1. Source of measurements M1 to M4 and M9 — the twelve columns and nine automatic indexes, the statement trace, the cost-against-unchanged-content probe with its read controls and payload sizes, the 8 000-byte indexed-string limit, the consistency levels, and the freshness decomposition. Conditions in the measurement appendix.
19. Measurement record, three-replica campaign: `findings_rf3.json` and its report, 2026-09-17, six-node two-datacentre ring with the working keyspace at three replicas. Source of M5, the Paxos round and its coordinator-side figures, and of M6, the Data API tier's startup configuration showing the consistency properties and their asymmetry.
20. Measurement record, nested shredding: raw CQL row read on the single node after a Data API insert, with filter probes and depth and width stress, 2026-09-17. Source of M8 — the dotted-path encoding, the array-of-objects limitation and the three shape limits.
21. Observed on the same stack on 4 September 2026, outside the campaigns above and therefore without their conditions record: `createTable` through the Data API answers `TABLE_FEATURE_NOT_ENABLED` on v1.0.33. Reported here because it bears on a design

option; it carries less evidentiary weight than the numbered measurements and should be re-checked on the build you deploy.

22. Measurement record, field-versus-byte separation: `findings_fieldbyte.json` and `probe_field_vs_byte.py` v1.0, 17 September 2026, three-replica ring in the disk-bound state established by note 44. Two series, two passes, thirty timed repetitions per configuration after five discarded warm-ups; verdict rule declared before execution. Source of measurement M11.
23. Measurement record, disk regime: `disk_state.json` and `disk_regime_driver.py`, 17 September 2026. Source of measurement M10 and of the proof that the regime was reached rather than assumed: 6.05 GiB on disk against a 2 048 MiB memtable budget, 40 SSTables after a major compaction, the compaction counter crossing from 651 to 729, and caches invalidated.
24. Measurement record, tier-versus-storage separation: `findings_tier.json` and `probe_tier_vs_storage.py` v1.0, 17 September 2026, three-replica ring in the same proven disk-bound regime as note 44. Two independent methods, two passes, thirty timed repetitions per size after five discarded warm-ups. Source of measurement M12.
25. Measurement record, read forced onto the SSTable path: `rmw_postflush.py`, 18 September 2026. Variant B of the cost series with a flush between the write and each mutation, so that the cycle's internal `SELECT` merges SSTables instead of reading the memtable. Thirty repetitions per size. Source of measurement M14, and of the requalification of the per-kilobyte rate in section 4.1.
26. Measurement record, cross-engine comparison: `probe_comparative.py` v1.0 with an added direct-CQL arm (`hcd_cql_arm.py`), 18 September 2026, all arms on one host in one session with a matched series and repetition count; the merge step verified the host fingerprint and series shape before emitting any ratio. Source of measurement M13, and of the tier-share estimate that disagrees with M12.
27. Measurement record, read and search axes: `probe_read_search.py`, 18 September 2026, one million documents per arm on one host, MongoDB as a three-member replica set at majority write concern, cache-resident regime on both sides. Fifty repetitions per measurement. Source of measurements M15 and M16.
28. IBM, *MongoDB Enterprise Advanced with IBM*, product page, and IBM Support, *MongoDB Enterprise Advanced with IBM 7.0.x* lifecycle page, which states that the 7.0.x offering is generally available as of 29 April 2025, can be purchased and is fully supported by IBM. <https://www.ibm.com/products/mongodb-enterprise-advanced> and <https://www.ibm.com/support/pages/mongodb-enterprise-advanced-ibm70x>

29. Measurement records, aggregation and counting: `findings_agg_hcd.json`, `findings_agg_mongodb.json`, `findings_agg_cqlref.json`; probes `probe_aggregation.py` and `agg_cql_arm.py`, 18 September 2026, one host, 200 000 documents in ten categories, MongoDB 8.0.32 as a three-member replica set at majority write concern with journalling, HCD 2.0.6 with Data API v1.0.33 at three replicas. Source of measurements M20 to M23. These four were published before receiving an adversarial pass; the pass is in the repository as the campaign-7 challenge document, and its findings are folded into the appendix entries.
30. DataStax, *Count documents*, Hyper-Converged Database API reference, 1.2 line: states that exact counting is a slow, expensive operation, that a count exceeding the limit set by the caller or by the API returns an indication of the limit instead of the count, and recommends the estimated count or the bulk loader for large volumes. The companion Astra DB reference for the same method states that exact counting of an arbitrary number of documents is not supported by the Data API. Scope caveat: the 1.2 page, the 2.0 page having the same rendering problem noted in section 3. <https://docs.datastax.com/en/hyper-converged-database/1.2/api-reference/document-methods/count-all.html>
31. Repository, `data/derived/inference.json` and `docs/STATISTICS.md`: a re-analysis of every latency comparison in the dossier from the summary statistics that survive — the raw per-observation series having been discarded by every probe but one — asking whether the observed ranges of the two arms are disjoint. Where they are, every observation of one arm beat every observation of the other, and the direction is certain without a distributional assumption; the exact one-sided permutation probability of that outcome is $1/C(n_A+n_B, n_A)$. Ninety-three comparisons; fifty-six disjoint, thirty-four overlapping. Source of every “certain” and “not established” in the table of section 4.4, and of the finding that the comparative harness fitted its slopes on pass one only. <https://github.com/davidleconte/shredding-cost-hcd-mongodb>
32. OpenCredo, *Patterns of Successful Cassandra Data Modelling*. States that the engine supports no complex processing at query time — filtering, joins, search, pattern matching or aggregation — and that data must be stored partitioned, sorted, denormalised and filtered in advance, the goal being to store pre-computed answers to the questions the application will ask, with roughly a table per query. A practitioner source rather than a vendor page; it states the doctrine the vendor’s own modelling documentation teaches. <https://opencredo.com/blogs/cassandra-data-modelling-patterns>

33. DataStax Community, *Are querying multiple partitions or multiple tables an anti-pattern in Cassandra?*. States that reading multiple partitions is not necessarily an anti-pattern but affects cluster performance, the coordinator becoming overloaded as it fans out requests; tolerable for two to five partitions and bad for the application and the cluster beyond that. A community answer on the vendor's own forum, which is weaker provenance than a documentation page, and the claim is graded accordingly.
<https://community.datastax.com/questions/5599/cassandra-anti-pattern-in-logical-data-model-query.html>

Appendix — sources

Each entry records what the source establishes, the edition or version to which it pertains, and the locator. All were consulted on 17 September 2026 unless otherwise stated. Numbering matches the notes above and is continuous across both parts.

1 — Stargate, *data-api* configuration record

Establishes: upstream defaults for page size (50), insert count (20), indexes per collection (10), indexes per database (50), LWT client retries (3).

Version: branch `main`, upstream open-source project — *not* the shipped HCD product.

<https://github.com/stargate/data-api/blob/main/CONFIGURATION.md>

2 — DataStax, Data API limits

Establishes: page size 20; maximum insertions per transaction 100.

Version: Astra DB Serverless, same Data API surface. Scope caveat: the HCD 2.0 limits page could not be rendered during verification.

<https://docs.datastax.com/en/astra-db-serverless/api-reference/dataapi-limits.html>

3 — DataStax Mission Control, HCD guardrails

Establishes: the `sai_indexes_total` guardrail set to 100.

Version: HCD product guardrail configuration.

<https://docs.datastax.com/en/mission-control/administration/data-plane/configure-hcd-guardrails.html>

5 — DataStax, Cassandra drivers overview

Establishes: the base engine version of the distribution.

"HCD 2.x is based on Cassandra 5.0."

Version: HCD 2.0.

<https://docs.datastax.com/en/hyper-converged-database/2.0/drivers/cassandra-drivers-overview.html>

6 — DataStax, tables reference

Establishes: fixed-schema tables alongside schemaless collections.

"Tables are best for use cases where your data is structured and adheres to a pre-defined schema, known as a fixed schema."

Version: HCD. Caveat: tables via the Data API carry a public-preview designation.

<https://docs.datastax.com/en/hyper-converged-database/2.0/api-reference/tables.html>

7 — DataStax, HCD frequently asked questions

Establishes: the Data API as a schema-less, document-based interface over both structured and unstructured data.

Version: HCD 2.0.

<https://docs.datastax.com/en/hyper-converged-database/2.0/faqs.html>

8 — DataStax, capacity planning

Establishes: the two sizing bounds quoted in section 6.1.

"a good rule of thumb is to keep the number of rows per partition under 100,000 and the partition size under 100 MB."

Version: HCD 1.1. Scope caveat: the 2.0 data-model page could not be rendered; figures corroborated by entry 9.

<https://docs.datastax.com/en/hyper-converged-database/1.1/install/capacity-planning.html>

9 — DataStax, data-modelling best practices

Establishes, independently: the partition-size bound.

"DataStax recommends keeping partitions small, ideally under 100 MB."

Version: CQL for HCD.

<https://docs.datastax.com/en/cql/hcd/data-modeling/best-practices.html>

10 — DataStax, list table metadata

Establishes: the JSON primary-key structure, including sort direction encoding.

```
"primaryKey": { "partitionBy": [ "user_id" ], "partitionSort": { "order_id": 1, "order_date": -1 } }
```

Version: Data API, Astra DB Serverless documentation; identical API surface.

<https://docs.datastax.com/en/astra-db-serverless/api-reference/table-methods/list-table-metadata.html>

11 — Stargate project, introducing the JSON API

Establishes: the shredding technique by name. Does *not* enumerate column names.

"we're using an improved approach known as 'super shredding'"

Date: 13 March 2023.

<https://stargate.io/2023/03/13/introduce-stargate-mongoose.html>

12 — Stargate project, the Cassandra Documents API

Establishes: the design intent regarding read-before-write, for the *earlier* Documents API.

"we resolved to avoid doing any read-before-write operations at all costs"

Date: 19 October 2020. Scope caveat: architecturally distinct from the current Data API; cited here precisely because it complicates the structural inference in section 4.

<https://stargate.io/2020/10/19/the-stargate-cassandra-documents-api.html>

13 — DataStax, lightweight transactions

Establishes: the Paxos phase structure of conditional writes, and the interaction hazards of mixing LWT with normal operations.

Version: HCD.

<https://docs.datastax.com/en/hyper-converged-database/1.1/architecture/database-internals/lightweight-transactions.html>

33 — DataStax, how transactions differ

Establishes: the absence of ACID transactions with rollback or locking; partition-level write atomicity; row-level isolation.

Version: HCD.

<https://docs.datastax.com/en/hyper-converged-database/1.1/architecture/database-internals/transactions-differ.html>

34 — MongoDB, Atomicity and Transactions

Establishes: the scope and the boundary of MongoDB's atomicity guarantee.

"In MongoDB, write operations are atomic on the single-document level, even if modifying multiple values." — and: "When a single write operation (e.g. db.collection.updateMany()) modifies multiple documents, the modification of each document is atomic, but the operation as a whole is not atomic."

Version: MongoDB Database Manual, current line.

<https://www.mongodb.com/docs/manual/core/write-operations-atomicity/>

35 — MongoDB, `$set` update operator

Establishes: targeted field modification rather than full document rewrite, stated comparatively against replacement.

"Targeted Updates: \$set modifies only the specified fields, ensuring efficient updates by avoiding unnecessary writes and overhead when you work with large documents."

Version: MongoDB Database Manual, current line; behaviour of empty operands noted from MongoDB 5.0.

<https://www.mongodb.com/docs/manual/reference/operator/update/set/>

36 — MongoDB, *Indexes*

Establishes: exactly one index is created without instruction, and it cannot be dropped.

"MongoDB creates a unique index on the `_id` field during the creation of a collection. The `_id` index prevents clients from inserting two documents with the same value for the `_id` field. You cannot drop this index."

Version: MongoDB Database Manual, current line.

<https://www.mongodb.com/docs/manual/indexes/>

37 — MongoDB, `createIndexes` reference

Establishes: the wildcard index as an opt-in mechanism for indexing arbitrary fields; cited here in the interest of comparative fairness.

"The created index can support queries on any arbitrary field within documents in the collection" — with the noted exclusion of `_id` by default.

Version: MongoDB Database Manual v8.0.

<https://www.mongodb.com/docs/v8.0/reference/command/createindexes/>

49 — IBM, MongoDB Enterprise Advanced with IBM

Establishes: that IBM sells and supports the comparator under its own name, the 7.0.x offering being generally available as of 29 April 2025 per IBM's lifecycle page. Bears on the disclosure only. Consulted 18 September 2026.

<https://www.ibm.com/products/mongodb-enterprise-advanced> ·

<https://www.ibm.com/support/pages/mongodb-enterprise-advanced-ibm70x>

53 — OpenCredo, patterns of successful Cassandra data modelling

Establishes the modelling doctrine this engine is built on: no complex processing at query time, data stored partitioned, sorted, denormalised and filtered in advance, and pre-computed answers rather than computed ones. Practitioner source, not a vendor page. Consulted 18 September 2026.

<https://opencredo.com/blogs/cassandra-data-modelling-patterns>

54 — DataStax Community, multi-partition queries as an anti-pattern

Establishes the coordinator cost of fanning out across partitions: tolerable for two to five, bad for the cluster beyond. A community answer on the vendor's forum; graded R rather than D on that account. Consulted 18 September 2026.

<https://community.datastax.com/questions/5599/cassandra-anti-pattern-in-logical-data-model-query.html>

Appendix B — measurements

Each entry records what was observed, on which build, under which conditions, and where the raw record lives. A measurement establishes what one build did on one day; it is not a vendor's documentation and does not license comparison with any other product. Identifiers M1 to M8 are the same identifiers used in the architecture decision record that accompanies the raw files, and they are continuous across both parts of this diptych.

Common conditions

IBM DataStax HCD 2.0.6, release string `5.0.7.0-ea50e91ba01f` read from the running node (`nodetool version`, `system.local`); Data API `stargateio/data-api` v1.0.33; astrapy 2.3.1. One node, one datacentre, RF 1 (`NetworkTopologyStrategy {dc1: 1}`), 16 vnodes. Virtual machine, 80 vCPU Intel Xeon Gold 6148 @ 2.40 GHz, 220 GiB RAM; the database container limited to 3 GiB with a 2 GiB heap; storage class of the backing media not determinable from the guest. `commitlog_sync periodic`, 10 000 ms. Working set memtable-resident throughout: no SSTable was flushed and no compaction crossed. Load driven at a fixed offered rate of 50 writes/s for 120 s, open loop, schedule slip under 0.06 s. Client on the same host over loopback. Harness `verify_storage_claims.py` 1.0 with two mechanical fixes (client environment set to HCD; probe-3 ballast excluded from indexing because the Data API refuses any indexed string above 8 000 bytes). 17 September 2026.

Full record: `findings.json`.

M1 — Physical column layout

Establishes: a collection created with default options and no vector dimension is one table of twelve columns, with nine storage-attached indexes created automatically over all but `key`, `tx_id` and `doc_json`. Read from `system_schema.columns` and `system_schema.indexes`; identical in two runs.

```
key frozen<tuple<tinyint,text>> · tx_id timeuuid · doc_json text · exist_keys set<text> ·
array_size map<text,int> · array_contains set<text> · query_bool_values
map<text,tinyint> · query_dbl_values map<text,decimal> · query_text_values
map<text,text> · query_null_values set<text> · query_timestamp_values
map<text,timestamp> · query_lexical_value text
```

M2 — The write path of a single-field update

Establishes: with query tracing at probability 1.0 on the node, each of ten `updateOne` operations carrying one `$set` produced, in order, a `SELECT` of the whole document by key and then one `UPDATE` rewriting every derived column and `doc_json`, conditioned on `tx_id`. Each `deleteOne` produced a `SELECT` of `key, tx_id` then `DELETE ... IF tx_id = ?`. Each `insertOne` produced one `INSERT ... IF NOT EXISTS`. Tracing was reset to 0.0 afterwards. Ten of ten updates showed the pair.

```
SELECT key, tx_id, doc_json FROM ... WHERE key = ? LIMIT 1 — then — UPDATE ...
SET tx_id = now(), exist_keys = ?, array_size = ?, array_contains = ?,
query_bool_values = ?, query_dbl_values = ?, query_text_values = ?,
query_null_values = ?, query_timestamp_values = ?, query_lexical_value = ?, doc_json
= ? WHERE key = ? IF tx_id = ?
```

M3 — Cost of a constant mutation against unchanged content

Establishes: with one scalar field mutated by `$set`, thirty timed repetitions after five warm-ups per size, and the `updateOne` HTTP payload constant at every size (122–124 bytes request, 47 bytes response), median latency grew 7.50× (12.1 ms → 90.8 ms) between a 1 KB document and a 128 KB document whose additional content was seventeen indexed string fields that the mutation never touched; a same-size read by key grew 1.56×. With the same total sizes but the additional content excluded from indexing, the update grew 1.92× and 2.21× in two runs against reads of 1.86× and 1.94× — indistinguishable from a read under the pre-registered control rule, and therefore not, on its own, evidence of the cycle that M2 observes directly.

Differenced at equal byte sizes, the indexing term is 2.3 ms at 8 KB, 17.3 ms at 32 KB and 68.7 ms at 128 KB. It is *not* divisible into a per-field rate: each ballast field was an 8 000-byte indexed string, so field count (1 → 17) and indexed byte volume ($\approx$ 8 KB → 136 KB) were grown in lockstep and neither share is recoverable from this run. A revision of this article reported such a rate; it is withdrawn. The experiment that would separate them — seventeen short indexed fields at one size, against one long indexed field at the same size — has not been run.

Raw record: `probe3_variant_b.json`, `control_read_B.json`, `control_read_A_rep2.json`. — published in the repository, github.com/davidleconte/shredding-cost-hcd-mongodb

M4 — Consistency levels

Establishes: every one of seventy traced statements against the collection ran at `LOCAL_QUORUM`; every one of thirty conditional statements carried `LOCAL_SERIAL` as its serial consistency level. The client set no level; these are the Data API tier's defaults. On one node with RF 1 a single replica satisfies both, so the level's name is established and its multi-replica cost is not.

Conditions for M5 to M8 — the three-replica ring and the nested-document read

M7, M8 and M9 were taken on a six-node, two-datacentre ring, the working keyspace at three replicas in one datacentre, one replica per rack; nodes larger than the single-node host used for M1 to M6 (8 GiB with a 4 GiB heap against 3 GiB with a 2 GiB heap), which means end-to-end latencies across the two campaigns mix a replication effect with a hardware difference and must not be compared directly. Coordinator-side figures and protocol traces are structural and do not carry that confound. M8 was read on the single node. Every regime memtable-resident; tracing raised to 1.0 and restored to 0.0 on every node touched; keyspaces dropped and snapshots cleared afterwards. 17 September 2026.

M5 — The price of the fifth operation, at three replicas

Establishes: every conditional write runs a full three-replica Paxos round. The coordinator reads the precondition, sends a prepare request to the other two replicas, both promise the ballot and return the read row, the coordinator proposes, both accept and append to commitlog and Paxos memtable, then commit. Coordinator-side median 13.7 ms for the conditional `UPDATE` against 6.6 ms for a non-conditional `SELECT`. The four verdicts established at one replica held unchanged at three.

Not established: a disk-bound equivalent, and a client-side rather than coordinator-side figure for the round.

Conditions for M10 and M11 — the disk-bound three-replica run

Same product and Data API version; three-replica ring, six nodes over two datacentres, nodes of 8 GiB with a 4 GiB heap, memtable budget 2 048 MiB. Regime established and evidenced rather than assumed: 24 576 documents of incompressible ballast, 6.05 GiB on disk, flush to 82 SSTables then a major compaction to 40, compaction counter 651 → 729, key and row caches invalidated. Storage class undeterminable from the guest and not invented: an ext4 filesystem on a virtual SCSI disk, effective sequential write 101 MB/s by `dd` with `fsync`; the physical medium was not established. 17 September 2026.

Declared mechanical correction. The driver first reported the regime as not reached, on the data-volume check alone. The cause was the ballast itself: a constant byte repeated is compressed roughly 120:1 by the table's compressor, so six logical gigabytes occupied fifty megabytes on disk. The ballast was changed to incompressible content and nothing else — no size, threshold, repetition count or proof obligation was touched. Two host-to-container adaptations were also declared. This is recorded here because a driver that had not checked its own precondition would have reported a disk-bound run that was nothing of the kind.

M10 — The coefficient under a disk-resident *dataset*, which is not a disk-resident *read*

Corrected. An earlier revision of this entry stated that the indexed-content coefficient does not collapse when the data no longer fits in memory. That overstated what was measured: the dataset was on disk, but the row the probe reads is the row it has just written, and that row is memtable-resident whatever surrounds it. Measurement M14 forced the read onto the SSTable path and the coefficient did collapse, from $\times 5.94$ to $\times 1.52$. What M10 establishes is narrower: memory pressure and cold caches, with the read still served from memory, leave the coefficient intact.

Observed: Variant B grew $\times 5.94$ (medians 20.4 $\rightarrow$ 121.2 ms across 1 KB to 128 KB) against $\times 7.50$ in the original memtable run; the unindexed variant grew $\times 1.78$ against a read control of $\times 1.89$, which is inconclusive under the pre-registered control rule exactly as it was in the first campaign. Index maintenance remained a first-order cost in that regime.

Not established: a disk-resident read, which M14 supplies; and the attribution of the $\times 7.50 \rightarrow \times 5.94$ difference. The two campaigns differ in regime, in replication factor and in node size at once.

Raw record: `disk_state.json` and the campaign report. — published in the repository, github.com/davidleconte/shredding-cost-hcd-mongodb

M11 — Which term of the indexing cost carries it

Establishes, by two series each holding one term fixed: at a constant 64 KiB of indexed content, raising the field count from 9 to 128 moved the median update by $\times 1.128$ and $\times 1.187$ in two passes; at a constant 16 fields, raising the bytes per field from 512 to 8 000 moved it by $\times 4.556$ and $\times 4.738$, with r^2 of 0.9999 in both. Least-squares slope and endpoint ratio agree to three significant figures. The derived rate is **0.763 and 0.790 ms per kilobyte of indexed content**, with a residual field term of 0.077 to 0.111 ms per field.

Internal control: the configuration of 16 fields $\times$ 4 096 bytes belongs to both series and was measured twice per pass; the readings agreed to 2.0 % and 0.7 %, against a pre-declared invalidation threshold of 10 %.

Not established, and reported rather than smoothed: the pre-registered verdict rule required a fields-series endpoint ratio at or below 1.15 to call bytes dominant. The passes returned 1.128 and 1.187, straddling it, so the formal verdict is inconclusive. Neither approaches the 1.5 that would make field count an independent driver. The exact size of the residual field term is therefore open.

Also not established: the per-kilobyte rate was measured only in the disk regime and not re-run in a memtable one.

Raw record: `findings_fieldbyte.json`. — published in the repository, github.com/davidleconte/shredding-cost-hcd-mongodb

M12 — Where the per-kilobyte rate is paid

Establishes, by two independent methods in the disk-bound regime of M10: the rate of roughly 0.77 ms per indexed kilobyte divides into a storage term of 0.547 ms/KiB (r^2 0.997) and a Data API tier term of 0.219 ms/KiB (r^2 0.98) — the tier being about 29 % of the rate. Their sum reproduces the total by construction and confirms nothing, the tier term being defined as total minus storage; an earlier revision of this appendix presented that identity as a replication and was wrong. The genuine replication is of the *total* slope across campaigns: 0.763 and 0.790 in M11, 0.772 here, on a different keyspace on a different day. A second method, subtracting coordinator-side trace durations from client-observed latency, puts the tier lower at 0.145 ms/KiB and 19 %.

Method: arm A, `updateOne` through the Data API; arm B, the identical CQL statement pair against the same row with the shredded values read back rather than recomputed, so the storage work is identical and no JSON is parsed. Thirty timed repetitions per size after five discarded warm-ups, two passes, whose tier slopes came back thirty per cent apart at 0.190 and 0.248. A third, weak and independent indicator: the tier container's CPU alternated between about 0.5 % while arm B ran and 40–230 % while arm A ran, so the tier does substantive computation rather than merely forwarding bytes.

Not established, and the reason no single split is published. The two methods agree to within a quarter at 8, 16 and 32 KiB and diverge beyond — 34 % at 64 KiB, 28 % at 125 KiB — because they subtract different things: the bypass removes a full client-side CQL execution, the trace only the coordinator's internal duration. The pre-declared tolerance was a quarter. Only the direction and the order of magnitude survive both, and the defensible range for the tier share is a fifth to a third rather than a point. Two further dispersions are unquantified: the thirty per cent spread between the bypass method's own passes, and the width of the direct-CQL distribution being subtracted, whose ninety-fifth percentile reaches three and a half times its median at the smallest size.

Also not established: the internal decomposition of the tier term, which is JSON parsing, shredding and response serialisation together; the share of that term attributable to client-library differences between the Data API client and the CQL driver; and any absolute figure transferable to a tier under a CPU limit, the container measured here having had none on an eighty-core host.

Declared mechanical correction: arm B initially bound the composite partition key as a simple statement parameter, which the server rejected; prepared statements were used instead, these carrying the column type. No threshold, size or repetition count was touched.

Raw record: `findings_tier.json` . — published in the repository, github.com/daviddleconte/shredding-cost-hcd-mongodb

M13 — The comparison, on one host

Establishes: with the identical mutation series, the same repetition counts and a declared durability mapping, all arms on one host in one session — slope per indexed kilobyte of 0.0098 for MongoDB with nothing indexed beyond `_id`, 0.0176 for MongoDB under a wildcard index over every field, 0.704 for the decomposing engine driven directly through CQL, and 0.778 for the same engine through the Data API. With the tier removed, against the capability-matched arm, the slope ratio is forty on pass one and 116 on pass two, and against the default arm seventy and eighty-six; on pass two the denominator's confidence interval reaches zero, so the ratio is unbounded by Fieller's theorem and no value of it is publishable. The ratio of *latencies*, which is what a reader hears, is 3.3× at 8 KB and 13.3× at 128 KB. No single multiplier is published; the defensible finding is a widening gap of roughly three- to thirteen-fold.

Method: the merge step compares the host fingerprint and the series shape of every record and refuses to emit a ratio unless they match. Durability mapped as majority write concern with journalling against `LOCAL_QUORUM` with a periodic commit-log sync — a mapping that favours the decomposing engine, which acknowledges before its sync while MongoDB waits for its journal.

Not established: any axis other than the mutation of one field. Filtered reads, vector search, aggregation and write-to-visible latency were not measured, and the last of these is where this platform's structural advantage lies. One host, one deployment shape, one workload.

Raw record: the comparative merge output. — published in the repository, github.com/daviddleconte/shredding-cost-hcd-mongodb

M14 — The rate's boundary condition

Establishes: forcing the read-modify-write cycle's internal `SELECT` onto the SSTable path, by flushing between the write and each mutation, collapses the growth with indexed volume from $\times 5.94$ to $\times 1.52$ and introduces a fixed cost of roughly ninety to a hundred milliseconds. The per-kilobyte rate is therefore a memtable-regime phenomenon; where the read leaves memory, the read path dominates and indexed volume ceases to be the driver.

Consequence for the record: measurement M10 was labelled as showing the coefficient holding under disk. It showed no such thing. The dataset was disk-resident; the probed row never was, a read-modify-write cycle re-reading what it has just written. M10 establishes that memory pressure and cold caches leave the coefficient intact, and nothing about leaving memory.

Not established, and the first item is a gap in the record rather than in the method: the post-flush run does not state its topology or replication factor, while the memtable figures it is compared against were taken on a three-replica ring, so the contrast may mix the read path with a deployment change. Then, in both directions: flushing before every mutation leaves one SSTable per flush, so the internal read merges an unnaturally fragmented set and the fixed cost is an upper bound; and the operating system page cache could not be dropped, so the SSTable path is not a cold one. A realistic disk-bound figure lies between the two regimes measured, and no measurement in this study occupies that middle.

Raw record: the post-flush probe output. — published in the repository,
github.com/davidleconte/shredding-cost-hcd-mongodb

M15 — The read and search axes, at a million documents

Establishes, all arms on one host with a million documents loaded and both sides cache-resident: filtering on a field for which no index was declared costs 376.7 ms on MongoDB as ordinarily deployed, which scans; 17.4 ms on the decomposing engine, which had indexed it automatically; and 0.73 ms on MongoDB under a wildcard index. Fetching one document by identifier costs 0.53–0.62 ms on MongoDB and 10.4 ms on the decomposing engine. Fifty repetitions per measurement, with both engines cache-resident throughout — a condition that measurement M14 shows to be decisive, so none of these figures should be read as a disk-regime result.

Reading: the automatic-indexing advantage is real against the default configuration, at twenty-two times, and absent against the capability-matched one, which is twenty-four times faster still. The point read is a straight loss of about twenty times, and its shape — a roughly fixed ten milliseconds — identifies it as the Data API tier on a path that involves no shredding, no rewrite and no consensus.

Not established: concurrency, multiple clients, multiple hosts, aggregation, or any workload other than the three single-operation axes measured. One host, one build of each engine, one regime.

Raw record: `findings_rs_hcd.json`, `findings_rs_mongo.json`. — published in the repository, github.com/davidleconte/shredding-cost-hcd-mongodb

M16 — Freshness on an ordinary secondary index: a tie, and the wrong object

Establishes: writing a document and then querying it through an ordinary secondary index, all three arms found it on the first attempt, forty times out of forty. Both engines maintain ordinary secondary indexes synchronously, so on the question of index *currency* there is parity. Elapsed time is not at parity: insert-to-found medians were 894 ms for MongoDB as ordinarily deployed, 33 ms for the decomposing engine and 7 ms for MongoDB under a wildcard index — twenty-seven times faster than the first, four and a half times slower than the second. A tie in the attempt histogram is not a tie in what a user waits for.

What it is not. The freshness claim this diptych makes for the decomposing engine concerns its *search and vector* index, which is synchronous, against MongoDB's search engine, which replicates asynchronously through change streams. That engine is not deployable on a plain replica set, being a preview component built for the vendor's managed service, so the axis could not be measured at all. A tie on ordinary indexes is parity on a different object, and must not be read as evidence against the claim — nor as evidence for it.

Raw record: as M15. — published in the repository, github.com/davidleconte/shredding-cost-hcd-mongodb

M20 — No server-side aggregation through the interface

Establishes: the Data API command allow-list on this build holds nineteen commands (`alterTable`, `countDocuments`, `createIndex`, `createTextIndex`, `createVectorIndex`, `deleteMany`, `deleteOne`, `estimatedDocumentCount`, `find`, `findAndRerank`, `findOne`, `findOneAndDelete`, `findOneAndReplace`, `findOneAndUpdate`, `insertMany`, `insertOne`, `listIndexes`, `updateMany`, `updateOne`); `aggregate`, `$group` and `distinct` answer `COMMAND_UNKNOWN`. A reading of the interface, independent of hardware, load and regime.

Doctrinal reading, which the measurement does not supply and which changes what the row means. The engine's own modelling doctrine tells the reader not to aggregate at query time but to store pre-computed answers, roughly a table per query; the architecturally correct route is therefore neither of the two measured, but an aggregate table maintained at write time. What the row establishes is a property of the *interface* — a document API whose proposition is comparator-like ergonomics and which lacks this verb — not a deficiency of the storage engine.

Not established: the same for the Data API's table mode, not tried; or for any other version, an allow-list being a product decision. Challenged as D11 and unchanged.

Raw record: `findings_agg_hcd.json` — published in the repository, github.com/daviddleconte/shredding-cost-hcd-mongodb

M21 — Exact counting is capped, and the cap is documented behaviour

Establishes: `countDocuments` on 200 000 documents, filtered and unfiltered, refused with `TooManyDocumentsToCountException: Document count exceeds 1000` on a stock container carrying no count-limit setting, despite a client upper bound of 400 000.

Reframed by the adversarial pass (D8): the vendor documents that exact counting beyond an API-set limit is not supported and returns an indication of the limit instead; the mechanism is therefore **D** and only the value of the threshold is **M**. This relocates the criticism from the ceiling, which is declared, to the recommended substitute, which is M22.

Also observed: MongoDB's exact count is itself a scan — 115 ms unfiltered, 157 ms filtered without an index, 12.7 ms once an index on the field was declared. It wins this axis because it answers, not because it answers cheaply.

Raw record: `findings_agg_hcd.json`, `findings_agg_mongodb.json` — published in the repository, github.com/daviddleconte/shredding-cost-hcd-mongodb

M22 — The estimator answers zero before the first flush

Establishes: `estimatedDocumentCount()` returned 0 at a median of 6.9 ms for a collection holding 200 000 documents, read minutes after loading and before any flush. The estimate is drawn from SSTable metadata, of which there is none until the memtable is flushed. Nothing in the response signals the condition. The same call on MongoDB returned 200 000 at 0.4 ms on this corpus.

Scoped by an adversarial pass, then un-scoped by measurement. Challenge D9 read the zero as a cold-start property and asked for the steady state. Campaign 7bis supplied it and refuted the challenge: 171 267 after a flush, 172 132 after a major compaction, against 200 000 — about fourteen per cent, not converging. The zero is the extreme of a permanent defect rather than a window, and this entry was understated rather than overstated. The zero remains direct evidence that campaign 7 ran cache-resident; 7bis then measured the same axis after flush and compaction, where the scan costs 130 127.0 ms, 2.1 % less than in cache — so the regime was not what carried the result.

Raw record: `findings_agg_hcd.json` — published in the repository, github.com/davidleconte/shredding-cost-hcd-mongodb

M23 — What aggregation costs by the only route available

Establishes: summing a field by category over 200 000 documents in ten groups, all arms verified exact against precomputed ground truth — MongoDB server-side `$group` median 222.7 ms ($n = 15$); the Data API client-side scan-and-sum median 133 984 ms ($n = 3$, spread 0.4 %) — **replayed at $n = 15$ by campaign 7bis, where the median is 132 985.311 ms, -0.7 %**, so the small sample was representative of the median; native CQL over a table designed for the query, 2 011 ms by per-partition sweep and 2 568 ms by cross-partition `GROUP BY` ($n = 15$ each). Ratios: 602× between MongoDB and the interface **at ten groups** — re-measured by campaign 7bis at three shapes: 584× at 10 groups [577, 592], 151× at 100 000 groups [148, 153], 609× filtered [599, 617], supports disjoint; 9.0× between MongoDB and the engine's best native route; 66.6× between the engine's two routes.

Decomposed by the adversarial pass (D7), and strengthened by it: the vendor's documented page size of 20 makes the scan 10 000 round trips; and 133 984.229 ms over 10 000 pages is **13.398 ms per page**, self-contained, with no residue. The first form of this decomposition imported the 10.4 ms point read of M15 — a different operation on a different collection from a different campaign — and produced a 30 s remainder that was an artefact of that import; `docs/THREATS-T0-VALIDITY.md` entry D7 records the error and the correction. The figure is a documented limit multiplied by a measured per-call cost, not tunable by the client. `U` as a derivation.

Not established: the native CQL route as a remedy for the document model, since it requires a table partitioned by the group key in advance; the index state under which the MongoDB `$group` ran, which the record does not state (D10); anything under concurrency or outside the cache regime; and the $n = 3$ figure's tail percentiles, which carry no information (D6) — that half of the reserve stands, while the reserve on the *median* falls, campaign 7bis having replayed it at $n = 15$ — the direction survives a factor-of-two error with hundreds to spare.

Raw record: `findings_agg_hcd.json`, `findings_agg_mongodb.json`,
`findings_agg_cqlref.json` — published in the repository, github.com/davidleconte/shredding-cost-hcd-mongodb

What the measurements do not establish

Any read that provably missed the operating system page cache: the cache was not dropped, root on the host being unavailable, and the dataset fits in host memory. The internal decomposition of the tier term: the subtraction removes JSON parsing, shredding and response serialisation together and separates none of the three from the others. A precise tier share, the two methods diverging beyond a quarter at the two largest sizes. A tier under a CPU limit, this one having had none. The attribution of the coefficient difference between campaigns, which mixes regime, replication factor and node size. The per-kilobyte rate outside a disk-bound regime. Anything about another version of HCD or the Data API, about Apache Cassandra 5.0 or 6.0, or about MongoDB.

Limitations

Stated here rather than buried, because work that grades its claims owes the reader an account of where the grading came out badly. These limitations apply to the diptych as a whole.

Four substantive claims could not be verified against sources of acceptable rank, and three of the four concern the decomposing engine rather than its comparator: the enumeration of generic column names, the read-modify-write cycle, the consistency levels at which Data API writes execute, and the current general-availability status of MongoDB search per edition. The first three were subsequently measured and hold on the builds named; they carry the **M** marker, which is weaker than **D**: an observation of a build is not a vendor's commitment. The fourth has since been resolved from the vendor's own pages; Part II carries the correction.

That the unverified residue falls almost entirely on one side is itself a finding, and it cuts against the side I work for. MongoDB's mutation semantics, its index policy and its asynchronous search path are documented in version-pinned reference material and were confirmed verbatim. The equivalent statements about the Data API's physical representation are not published anywhere I could reach. A reader may reasonably conclude that the system I know better is the one whose mechanism is harder to verify, and may weight the comparison accordingly. An earlier revision added that the asymmetry ran further still, nine measurements having been taken on one side and none on the other. That is no longer the case: section 4.4 reports the identical series run against both engines on one host. The documentary asymmetry above remains; the measurement asymmetry does not.

The four ratios most likely to be quoted are the least supported. The 44×, 79×, 40× and 72× are quotients of fitted slopes. The distribution-free apparatus that certifies the rest of this work compares pairs of samples and fits nothing, so it reaches none of them. Fitted properly, with the standard errors carried through, the denominator is not distinguishable from zero on the second of the two passes and the ratio is unbounded there; the point estimate moves by a factor of 2.6 between passes. Section 4.4 states this. A reader who takes one number from this diptych should take a latency range, never a slope ratio.

No confidence interval can be computed from this study by anyone, including its author. Every probe but one discarded the per-observation series and kept only the summary — count, minimum, median, ninety-fifth and ninety-ninth percentiles, maximum, standard deviation. What survives permits exactly one distribution-free question: whether the two arms' ranges are disjoint. Where they are, the direction is certain and an exact permutation probability follows; where they are not, the direction is not established from these data, whatever the medians suggest. Section 4.4 carries the answer for every axis. Two of the three margins that favour the decomposing engine do not survive it as latency comparisons; one of them survives as a proportion.

The shape of the cost is now resolved in one direction and not the other. The byte term is established across two passes with an internal control agreeing to under two per cent; the residual field term is not pinned down, the two passes straddling the pre-registered threshold at 1.128 and 1.187. The separation of the Data API tier's parsing cost from the storage engine's has since been done and

is reported in section 4.3, with the qualification that its two methods diverge at the larger sizes and only a range is defensible. The per-kilobyte rate was also measured only in the disk regime; the near-perfect linearity is expected to survive a memtable regime but was not re-run there.

The disk-bound caveat has been discharged, and replaced by a narrower one. An earlier revision noted that every regime was memtable-resident and that the engine described never touched its disk. A three-replica run in a proven disk-bound state — 6.05 GiB on disk against a 2 048 MiB memtable budget, forty SSTables, seventy-eight compactions crossed, caches invalidated — has since been executed.⁴⁴ The indexed-content coefficient did not collapse there: $\times 5.94$ against $\times 7.50$ in the original memtable run.

M That sentence, as an earlier revision left it, claimed more than the run supported, and the correction matters enough to state rather than quietly absorb. The dataset was on disk; the row the probe read was not. A read-modify-write cycle re-reads what it has just written, and that row sits in the memtable whatever surrounds it. The campaign proved a disk-resident *dataset*, never a disk-resident *read*. When the read is actually forced onto the SSTable path, the coefficient does collapse, from $\times 5.94$ to $\times 1.52$, and section 4.1 reports what replaces it. What that campaign established is narrower than its label: memory pressure and cold caches do not destroy the coefficient. Leaving memory does.

What remains is a page-cache confound and a hot-row confound, and they matter. The operating system page cache was not dropped, root on the host being unavailable, and the 6 GiB dataset fits comfortably in the host's memory: "disk-bound" here means the data genuinely exceeds the memtable budget and lives in SSTables, not that any given read missed RAM. Further, the read-modify-write probe re-reads the row it has just written, and that row is memtable-resident whatever the size of the dataset around it. The disk regime therefore acts through memory pressure and cold caches rather than by serving the probed row from disk. Anyone reading these figures as fully disk-resident latencies is reading more into them than was measured.

Two product pages could not be rendered during verification — the HCD 2.0 Data API limits page and the HCD 2.0 data-model page — so several product figures rest on the equivalent commercial documentation for the same API surface rather than on the exact edition under discussion. These substitutions are marked in the appendix entry concerned.

Everything concerning Cassandra 6.0 describes software in alpha, explicitly designated by its publishers as unsuitable for anything beyond development and testing. Its configuration surface, its syntax and its feature scope may all change before general availability.

Two sources were consulted through list mirrors rather than the Apache archive directly, and one through the project's pull-request traffic rather than published documentation. Where that provenance bears on the weight of a claim, the claim carries the **R** marker rather than **D** — with one exception I would rather name than let a reader find. The alpha classification quoted in Part II is marked **D** although the only reachable copy of that announcement was a list mirror. The quotation is verbatim and the tag dates corroborate it from the project repository itself, but by the rule stated in this very paragraph it ought to carry **R**. Consistency is a property of a system of markers, not of any one of them, and this is where mine failed.

